

Estrutura e função do processador

- 12.1** Organização do processador
- 12.2** Organização dos registradores
 - Registradores visíveis ao usuário
 - Registradores de controle e estado
 - Exemplo de organização de registradores de microprocessadores
- 12.3** Ciclo da instrução
 - Ciclo indireto
 - Fluxo de dados
- 12.4** Pipeline de instruções
 - Estratégia do pipeline
 - Desempenho do pipeline
 - Hazards do pipeline
 - Lidando com desvios
 - Pipeline de Intel 80486
- 12.5** Família de processadores x86
 - Organização dos registradores
 - Processamento de interrupção
- 12.6** Processador ARM
 - Organização do processador
 - Modos do processador
 - Organização dos registradores
 - Processamento de interrupção
- 12.7** Leitura recomendada

PRINCIPAIS PONTOS

- Um processador inclui tanto os registradores visíveis ao usuário como os registradores de controle/estado. Os primeiros podem ser referenciados, implícita ou explicitamente, em instruções de máquina. Os registradores visíveis ao usuário podem ser de uso geral ou de uso específico, como para armazenamento de números de ponto fixo ou ponto flutuante, endereços, índices e ponteiros de segmento. Os registradores de controle e estado são usados para controlar a operação do processador. Um exemplo óbvio é o contador de programa (*program counter*). Outro exemplo importante é palavra de estado do programa (PSW, do inglês *program status word*) que contém uma série de bits de estado e condição. Isso inclui os bits para refletir o resultado da mais recente operação aritmética, bits que habilitam interrupção e um indicador para saber se o processador está executando no modo supervisor ou usuário.
- Os processadores usam pipeline de instruções para acelerar a execução. Basicamente, o pipeline envolve quebrar o ciclo da instrução em um número de estágios separados que ocorrem em sequência, como ler a instrução, decodificar a instrução, determinar o endereço do operando, ler os operandos, executar a instrução e escrever o resultado do operando. As instruções se movem por esses estágios como em uma linha de montagem, então cada estágio pode trabalhar em uma instrução diferente ao mesmo tempo. A ocorrência de desvios e dependências entre instruções complicam o projeto e uso de pipelines.

Este capítulo discute os aspectos do processador que não foram cobertos na Parte Três e nos prepara para uma discussão sobre as arquiteturas RISC e superescalares nos capítulos 13 e 14.

Começamos com um resumo sobre a organização do processador. Depois analisaremos os registradores, os quais formam a memória interna do processador. Estaremos então aptos para retornar à discussão (iniciada na Seção 3.2) sobre o ciclo da instrução. Uma descrição do ciclo da instrução e uma técnica comum conhecida como *pipelining* de instruções completam a nossa descrição. O capítulo termina com uma análise de alguns aspectos das organizações x86 e ARM.



12.1 Organização do processador

Para entender a organização do processador, vamos considerar os requisitos que lhe são exigidos:

- **Buscar instrução:** o processador lê uma instrução da memória (registrador, cache, memória principal).
- **Interpretar a instrução:** a instrução é decodificada para determinar qual ação é requerida.
- **Obter os dados:** a execução de uma instrução pode requerer leitura de dados da memória ou um módulo de E/S.
- **Processar os dados:** a execução de uma instrução pode requerer efetuar alguma operação aritmética ou lógica com os dados.
- **Gravar os dados:** os resultados de uma execução podem requerer gravar dados para memória ou um módulo E/S.

Para fazer essas coisas, deve estar claro que o processador precisa armazenar alguns dados temporariamente. Ele deve lembrar a posição da última instrução executante para que possa saber onde obter a próxima instrução a ser executada. Ele precisa armazenar instruções e dados temporariamente enquanto uma instrução está sendo executada. Em outras palavras, o processador precisa de uma pequena memória interna.

A Figura 12.1 é uma visão simplificada do processador, indicando a sua conexão com o restante do sistema por meio do barramento do sistema. Uma interface similar seria necessária para qualquer estrutura de interconexão entre as descritas no Capítulo 3. O leitor irá lembrar que os principais componentes do processador são uma *unidade lógica e aritmética* (ALU) e uma *unidade de controle*. A ALU faz os cálculos ou processamento de dados de fato. A unidade de controle controla a movimentação de dados e das instruções que entram e saem do processador e controla a operação de ALU. Além disso, a figura mostra uma memória interna pequena que consiste de um conjunto de locais de armazenamento chamados de *registradores*.

A Figura 12.2 é uma visão um pouco mais detalhada do processador. Os caminhos de transferência de dados e controle lógico são destacados, inclusive um elemento chamado *barramento interno do processador*. Este elemento é necessário para transferir dados entre vários registradores e ALU, porque a ALU na verdade opera apenas os dados que estejam na memória interna do processador. A figura mostra também os elementos básicos típicos dela. Observe a semelhança entre a estrutura interna do computador como um todo e a estrutura interna do processador. Em ambos os casos, existe um pequeno conjunto de elementos principais (computador: processador, E/S, memória; processador: unidade de controle, ALU, registradores) conectados por caminhos de dados.

Figura 12.1 CPU com barramento de sistema

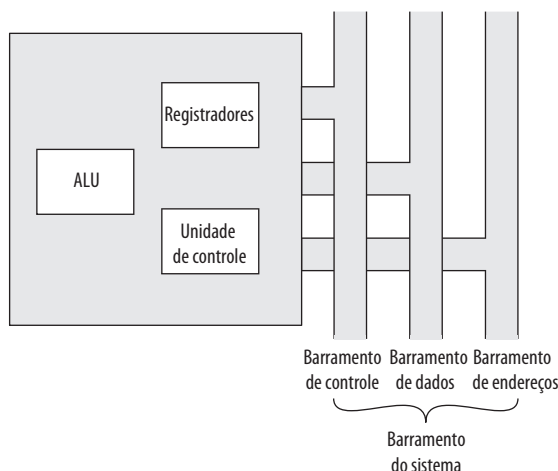
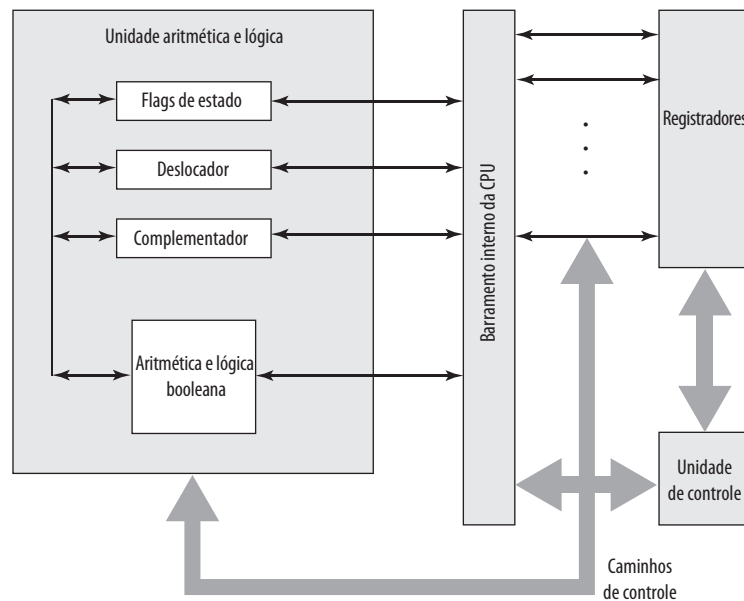


Figura 12.2 Estrutura interna da CPU

12.2 Organização dos registradores

Conforme discutimos no Capítulo 4, um sistema de computador emprega uma hierarquia de memória. Em níveis mais altos da hierarquia, a memória é mais rápida, menor e mais cara (por bit). Dentro do processador, existe um conjunto de registradores que funcionam como um nível de memória acima da memória principal e da cache dentro desta hierarquia. Os registradores no processador desempenham dois papéis:

- **Registradores visíveis ao usuário:** possibilitam que o programador de linguagem de máquina ou assembly minimize as referências à memória, pela otimização do uso de registradores.
- **Registradores de controle e estado:** usados pela unidade de controle para controlar a operação do processador e por programas privilegiados do Sistema Operacional para controlar a execução de programas.

Não há uma separação clara de registradores nessas duas categorias. Por exemplo, em algumas máquinas, o contador de programas é visível ao usuário (por exemplo, o x86) mas, em muitos outros, não é. No entanto, para o propósito da discussão que segue, usaremos essas categorias.

Registradores visíveis ao usuário

Um registrador visível ao usuário é aquele que pode ser referenciado pelos recursos da linguagem de máquina que o processador executa. Podemos dividi-los em seguintes categorias:

- Uso geral.
- Dados.
- Endereços.
- Códigos condicionais.

Registradores de propósito geral podem ser atribuídos para uma variedade de funções pelo programador. Algumas vezes, seu uso dentro do conjunto de instruções é ortogonal para a operação. Isto é, qualquer registrador de uso geral pode conter um operando para qualquer *opcode*. Isso permite o verdadeiro uso dos registradores de propósito geral. No entanto, frequentemente existem restrições. Por exemplo, pode haver registradores dedicados para ponto flutuante e operações de pilha.

Em alguns casos, os registradores de propósito geral podem ser usados para funções de endereçamento (por exemplo, indireto por registradores, deslocamento). Em outros casos, existe uma separação clara ou parcial entre os registradores de dados e os de endereços. **Registradores de dados** podem ser usados apenas para guardar dados e não podem ser empregados para calcular o endereço de um operando. **Registradores de endereços** podem ser, de certa forma, de uso geral ou podem ser dedicados para um modo de endereçamento em particular. Os exemplos incluem o seguinte:

- **Ponteiros de segmento:** em uma máquina com endereçamento segmentado (veja a Seção 8.3), um registrador de segmento guarda o endereço base do segmento. Pode haver múltiplos registradores: por exemplo, um para o sistema operacional e um para o processo atual.
- **Registradores de índice:** estes são usados para indexar endereços e podem ser autoindexados.
- **Ponteiros de pilha:** se houver endereçamento de pilha visível ao usuário, então normalmente haverá um registrador dedicado que aponta para o topo da pilha. Isso permite o endereçamento implícito: ou seja, as instruções de pilha como *push*, *pop* e outras não precisam conter um operando de pilha explícito.

Existem várias questões de projeto a serem discutidas aqui. Uma questão importante é se devemos usar registradores somente de propósito geral ou se devemos especializar o seu uso. Nós já tocamos nessas questões em capítulos anteriores porque isso afeta o projeto do conjunto de instruções. Com a utilização de registradores específicos, normalmente pode estar implícito no *opcode* a que tipo de registrador um determinado especificador de operando se refere. O especificador de operando deve identificar apenas um registrador dentro de um conjunto de registradores específicos em vez de um conjunto de todos os registradores, o que economiza bits. Por outro lado, esta especialização limita a flexibilidade do programador.

Outra questão de projeto é o número de registradores a serem oferecidos, registradores de propósito geral ou de dados mais os de endereços. Isso afeta novamente o conjunto de instruções porque mais registradores requerem mais bits para especificadores de operandos. Conforme discutido anteriormente, algo entre 8 e 32 registradores parece ideal (LUNDE, 1977^a). Menos registradores resultam em mais referências de memória; mais registradores não reduzem de forma notável as referências de memória (por exemplo, veja WILLIAMS e STEVEN, 1990^b). No entanto, uma nova abordagem que encontra vantagem no uso de centenas de registradores é mostrada em alguns sistemas RISC e discutida no Capítulo 13.

Finalmente, temos a questão de tamanho do registrador. Registradores que guardam endereços obviamente precisam ter pelo menos o tamanho suficiente para guardar o maior endereço possível. Registradores de dados deveriam ser capazes de guardar valores da maioria de tipos de dados. Algumas máquinas permitem que dois registradores contínuos sejam usados em conjunto para guardar valores de tamanho duplo.

Uma categoria final de registradores, a qual é ao menos parcialmente visível ao usuário, guarda *códigos condicionais* (também chamados de *flags*). Códigos condicionais são bits definidos pelo hardware do processador como resultado das operações. Por exemplo, uma operação aritmética pode produzir um resultado positivo, negativo, zero ou fora da capacidade. Além do resultado que é guardado no registrador ou na memória, um código condicional também é definido. O código pode ser testado na sequência como parte de uma operação de desvio condicional.

Os bits de códigos condicionais são coletados em um ou mais registradores. Normalmente eles fazem parte do registrador de controle. Geralmente, as instruções de máquina permitem que esses bits sejam lidos por referência implícita, mas o programador não pode alterá-los.

Muitos processadores, inclusive aqueles baseados na arquitetura IA-64 e os processadores MIPS, nem sequer usam códigos condicionais. Em vez disso, as instruções de desvios condicionais especificam uma comparação para ser feita e atuam no resultado dessa comparação, sem armazenar um código condicional. A Tabela 12.1, baseada em DeRosa e Levy (1987^c), mostra as principais vantagens e desvantagens dos códigos condicionais.

Em algumas máquinas, uma chamada de sub-rotina resultará automaticamente no salvamento de todos os registradores visíveis ao usuário para serem restaurados no retorno. O processador efetua salvamento e restauração como parte da execução das instruções de chamada e retorno. Isso permite que cada sub-rotina use os registradores visíveis ao usuário independentemente. Em outras máquinas, é responsabilidade do programador salvar os conteúdos dos registradores visíveis ao usuário relevantes antes de uma chamada de sub-rotina, incluindo as instruções para este propósito dentro do programa.

Tabela 12.1 Códigos condicionais

Vantagens	Desvantagens
1. Como os códigos condicionais são definidos por instruções normais aritméticas ou de movimentação de dados, eles devem reduzir o número de instruções de comparação e teste (COMPARE, TEST) necessárias.	1. Códigos condicionais acrescentam complexidade, tanto para hardware como para software. Os bits dos códigos condicionais são frequentemente modificados de maneiras diferentes por instruções diferentes, tornando a vida do microprogramador e do projetista de compiladores mais difícil.
2. Instruções condicionais, como BRANCH, são simplificadas em relação a instruções compostas como TEST AND BRANCH.	2. Códigos condicionais são irregulares; normalmente eles não fazem parte do caminho principal de dados e, por isso, requerem conexões extras de hardware.
3. Códigos condicionais facilitam desvios múltiplos. Por exemplo, uma instrução TEST pode ser seguida de dois desvios, um para menor ou igual a zero e outro para maior que zero.	3. Frequentemente, máquinas com códigos condicionais precisam adicionar instruções especiais que não usam códigos condicionais para situações especiais de qualquer forma, como verificação de bits, controle de laços e operações atômicas de semáforos.
	4. Em uma implementação de pipeline, códigos condicionais requerem sincronização especial para evitar conflitos.



Registradores de controle e estado

Existe uma variedade de registradores do processador que são empregados para controlar a operação do processador. Grande parte deles, na maioria das máquinas, não é visível ao usuário. Alguns podem ser visíveis às instruções da máquina executadas no modo de controle ou de sistema operacional.

É claro que máquinas diferentes terão diferentes organizações dos registradores e usarão terminologia diferente. Mostramos aqui uma lista razoavelmente completa de tipos de registradores com uma breve descrição.

Quatro registradores são essenciais para execução das instruções:

- **Contador de programas (PC):** contém o endereço de uma instrução a ser lida.
- **Registrador da instrução (IR):** contém a instrução lida mais recentemente.
- **Registrador de endereço de memória (MAR):** contém o endereço de uma posição de memória.
- **Registrador de buffer de memória (MBR):** contém uma palavra de dados para ser escrita na memória ou a palavra lida mais recentemente.

Nem todos os processadores possuem registradores internos designados como MAR e MBR, mas é necessário algum mecanismo de buffer equivalente pelo qual os bits a serem transferidos ao barramento do sistema são processados e os bits a serem lidos do barramento de dados são armazenados temporariamente é necessário.

Normalmente, o processador atualiza o PC depois de ler cada instrução para que o PC sempre aponte para a próxima instrução a ser executada. Uma instrução de desvio ou salto também irá modificar o conteúdo de PC. A instrução lida é colocada em IR, onde o *opcode* e os especificadores de operando são analisados. Os dados são trocados com a memória com o uso de MAR e MBR. Em um sistema organizado com barramentos, MAR se conecta diretamente ao barramento de endereços e MBR se conecta diretamente ao barramento de dados. Registradores visíveis ao usuário, por sua vez, trocam dados com MBR.

Os quatro registradores mencionados são usados para movimentar dados entre o processador e a memória. Dentro do processador, os dados precisam ser apresentados à ALU para serem processados. Ela pode ter acesso direto ao MBR e aos registradores visíveis ao usuário. Alternativamente, pode haver outros registradores de buffer na vizinhança do ALU; esses registradores servem como registradores de entrada e saída para a ALU e para trocar dados com o MBR e com os registradores visíveis ao usuário.

Muitos modelos de processador incluem um registrador ou conjunto de registradores frequentemente conhecido como *palavra de estado do programa* (PSW), o qual contém as informações de estado. Normalmente a PSW contém códigos condicionais e outras informações de estado. Campos comuns ou flags incluem:

- **Sinal:** contém o bit de sinal do resultado da última operação aritmética.
- **Zero:** marcado quando o resultado é 0.

- **Carry:** marcado se uma operação resultou em transportar (adição) para empréstimo (subtração) de um bit de ordem maior. Usado para operações aritméticas de múltiplas palavras.
- **Igual:** marcado se uma comparação lógica resultou em igualdade.
- **Overflow:** usado para indicar sobrecarga aritmética.
- **Habilitar/desabilitar interrupção:** usado para habilitar ou desabilitar interrupções.
- **Supervisor:** indica se o processador está executando no modo supervisor ou usuário. Algumas instruções privilegiadas podem ser executadas apenas no modo supervisor e algumas áreas de memória podem ser acessadas apenas no modo supervisor.

Vários outros registradores relacionados com estado e controle podem ser encontrados em um determinado modelo de processador. Pode haver um ponteiro para um bloco de memória contendo informações adicionais de estado (por exemplo, blocos de controle de processo). Em máquinas que usam interrupções vetoradas, um registrador de interrupção vetorada pode ser fornecido. Se uma pilha é usada para implementar algumas funções (por exemplo, chamada de sub-rotinas), então um ponteiro de pilha de sistema é necessário. O ponteiro da tabela de página é usado em um sistema de memória virtual. Finalmente, registradores podem ser usados no controle de operações E/S.

Uma série de fatores influencia o projeto da organização dos registradores de controle e estado. Uma questão fundamental é o suporte ao sistema operacional. Certos tipos de informações de controle são específicas do Sistema Operacional. Se o projetista do processador tem o entendimento funcional do sistema operacional a ser usado, então uma parte da responsabilidade da organização dos registradores pode ser designada ao sistema operacional.

Outra decisão fundamental do projeto é a alocação da informação de controle entre registradores e memória. É comum dedicar algumas primeiras centenas (mais baixas) ou milhares de palavras de memória para propósitos de controle. O projetista deve decidir quanto da informação de controle deve estar nos registradores e quanto em memória. Aparece a comum negociação entre custos *versus* velocidade.



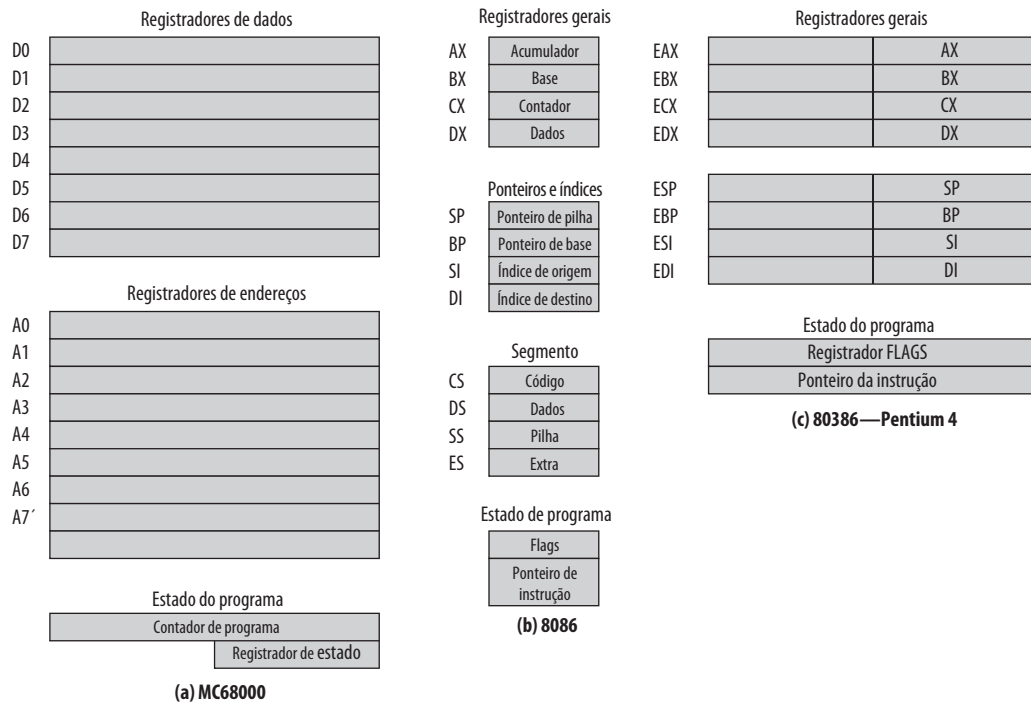
Exemplo de organização de registradores de microprocessadores

É instrutivo examinar e comparar a organização de registradores entre sistemas semelhantes. Nesta seção, iremos analisar dois microprocessadores de 16 bits que foram projetados quase ao mesmo tempo: Motorola MC68000 (STRITTER e GUNTER, 1979^d) e Intel 8086 (MORSE, POHLMAN e RAVENEL, 1978^e). As Figuras 12.3a e 12.3b ilustram a organização de registradores de cada um deles; registradores totalmente internos, como um registrador de endereços de memória, não são mostrados.

O MC68000 divide seus registradores de 32 bits em oito registradores de dados e nove registradores de endereços. Os oito registradores de dados são usados principalmente para manipulação de dados e também no endereçamento como registradores indexadores. O tamanho dos registradores permite operações de dados de 8, 16 ou 32 bits determinadas por *opcode*. Os registradores de endereço contêm endereços de 32 bits (sem segmentação); dois desses registradores são usados também como ponteiros da pilha, um para usuários e outro para o sistema operacional, dependendo do atual modo de execução. Os dois registradores são numerados como 7, porque apenas um pode ser usado por vez. O MC68000 inclui também um contador de programa de 32 bits um registrador de estado de 16 bits.

A equipe da Motorola quis um conjunto de registradores muito regular, sem nenhum registrador de uso especializado. A preocupação com a eficiência do código o levou a dividir os registradores em dois componentes funcionais, economizando um bit em cada especificador de registrador. Este parece ser um compromisso razoável entre generalização total e compactação de código.

O Intel 8086 usa uma abordagem diferente para organização de registradores. Cada registrador é de uso específico, embora alguns registradores possam ser usados como registradores de uso geral. O 8086 contém quatro registradores de dados de 16 bits que podem ser endereçados como um byte ou como 16 bits e quatro registradores indexadores e ponteiros de 16 bits. Os registradores de dados podem ser de propósito geral em algumas instruções. Em outras, os registradores são usados implicitamente. Por exemplo, uma instrução de multiplicação sempre usa o acumulador. Os quatro registradores de ponteiro também são usados implicitamente em várias operações; cada um contém um offset de segmento. Existem também quatro registradores de segmento de 16 bits. Três dos quatro registradores de segmento são usados de forma dedicada e implícita para apontar o segmento da instrução atual

Figura 12.3 Exemplo das organizações de registradores do microprocessador

(útil para instruções de desvio), um segmento contendo os dados e um segmento contendo pilha, respectivamente. Este uso dedicado e implícito possibilita a compactação de código com o custo da redução da flexibilidade reduzida. O 8086 inclui também um ponteiro de instrução e um conjunto de flags de 1 bit de estado e controle.

O objetivo desta comparação deve ser claro. Não há uma filosofia universalmente aceita no que diz respeito à melhor forma para organizar registradores de um processador (TOONG e GUPTA, 1981¹). Como acontece com o projeto do conjunto de instruções e tantas outras questões sobre o projeto dos processadores, tudo isso é ainda uma questão de gosto e julgamento.

Outro ponto instrutivo a respeito do projeto da organização de registradores está ilustrado na Figura 12.3c. Esta figura mostra a organização de registradores visíveis ao usuário para Intel 80386 (EL-AYAT e AGARWAL, 1985⁹), o qual é um microprocessador de 32 bits projetado como uma extensão do 8086.¹ O 80386 usa registradores de 32 bits. No entanto, para permitir a compatibilidade de programas escritos para máquinas anteriores, o 80386 mantém a organização de registradores original embutida na nova organização. Dada essa limitação do projeto, os projetistas dos processadores de 32 bits tiveram flexibilidade limitada ao projetar a organização dos registradores.



12.3 Ciclo da instrução

Na Seção 3.2 descrevemos o ciclo da instrução do processador (Figura 3.9). Para relembrar, um ciclo de instrução inclui os seguintes estágios:

- **Buscar:** lê a próxima instrução da memória para dentro do processador.
- **Executar:** interpreta *opcode* e efetua a operação indicada.
- **Interromper:** se as interrupções estão habilitadas e uma interrupção ocorre, salva o estado do processo atual a atende a interrupção.

¹ Como o MC68000 já usava registradores de 32 bits, MC68020 (MACDOUGALL, 1984^b) o qual tem uma arquitetura totalmente de 32 bits, usa a mesma organização de registradores.

Estamos na posição agora de poder elaborar algo a mais no ciclo da instrução. Primeiramente temos que introduzir um estágio adicional, conhecido como ciclo indireto.



Ciclo indireto

Vimos no Capítulo 11 que a execução de uma instrução pode envolver um ou mais operandos na memória, onde cada um deles requer um acesso à memória. Além disso, se o endereçamento indireto é usado, então acessos adicionais à memória são necessários.

Podemos pensar em obter um endereço indireto como sendo mais um estágio da instrução. O resultado é mostrado na Figura 12.4. A linha principal da atividade consiste em alternar as atividades de buscar a instrução e executar a instrução. Depois que uma instrução é lida, ela é examinada para determinar se algum endereçamento indireto está envolvido. Se estiver, os operandos necessários são obtidos usando endereçamento indireto. Durante a execução, uma interrupção pode ser processada antes de obter a próxima instrução.

Outra maneira de ver este processo está mostrada na Figura 12.5, a qual é uma versão revisada da Figura 12.4. Isso ilustra mais corretamente a natureza do ciclo da instrução. Uma vez lida a instrução, os seus especificadores de operandos devem ser identificados. Cada operando de entrada na memória é, então, lido na memória e este processo pode requerer endereçamento indireto. Operandos baseados em registradores não precisam ser obtidos da memória. Uma vez o *opcode* executado, um processo semelhante pode ser necessário para armazenar o resultado na memória principal.



Fluxo de dados

A sequência exata de eventos durante um ciclo de instrução depende do modelo do processador. No entanto, podemos indicar de uma maneira geral o que deve acontecer. Vamos supor um processador que emprega um registrador de endereço de memória (MAR), um registrador de buffer de memória (MBR), um contador de programa (PC) e um registrador de instrução (IR).

Durante o *ciclo de leitura*, uma instrução é lida da memória. A Figura 12.6 mostra o fluxo de dados durante esse ciclo. PC contém o endereço da próxima instrução a ser lida. Esse endereço é movido para MAR e colocado no barramento de endereços. A unidade de controle requer uma leitura de memória e o resultado é colocado no barramento e copiado para MBR e depois movido para IR. Enquanto isso, o PC é incrementado por 1, preparando-se para próxima leitura.

Uma vez terminado o ciclo de leitura, a unidade de controle examina o conteúdo de IR para determinar se ele contém um especificador de operando que use endereçamento indireto. Se for esse o caso, um *ciclo indireto* é efetuado. Conforme mostrado na Figura 12.7, este é um ciclo simples. Os *N* bits da extrema direita de MBR, o qual contém a referência de endereço, são transferidos para MAR. Depois, a unidade de controle requisita uma leitura de memória para obter o endereço desejado do operando em MBR.

Figura 12.4 O ciclo da instrução

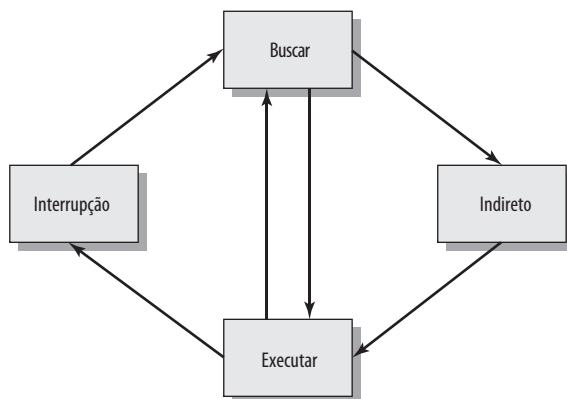
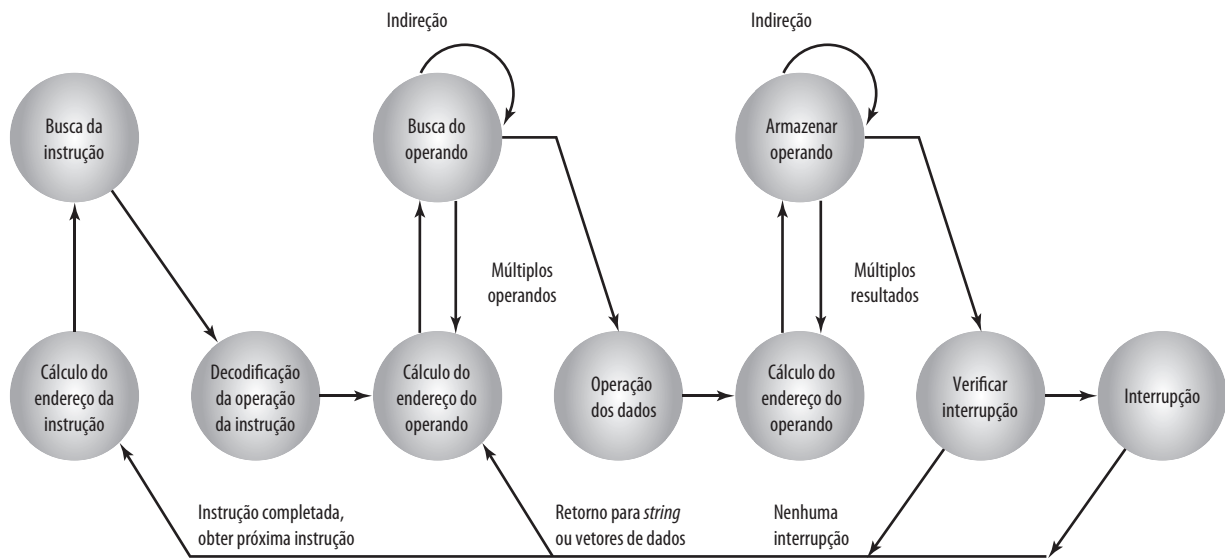
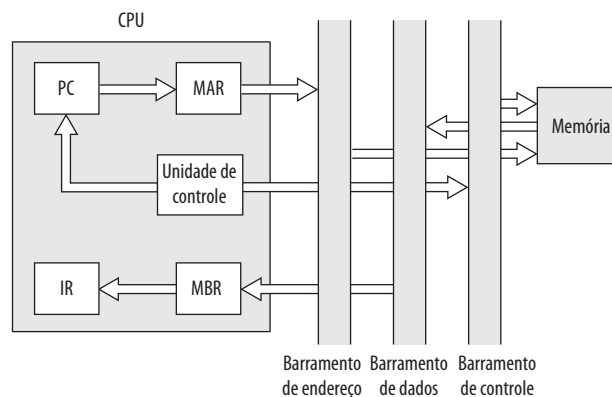


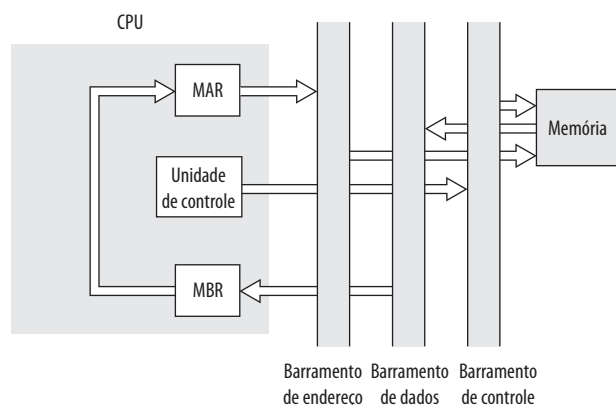
Figura 12.5 Diagrama de estado do ciclo da instrução

Os ciclos de leitura e indiretos são simples e previsíveis. O *ciclo de execução* assume muitas formas; a forma depende de qual das várias instruções de máquina está em IR. Este ciclo pode envolver a transferência de dados entre registradores, leitura ou escrita de memória ou E/S e/ou a utilização de ALU.

Assim como o ciclo de leitura e o indireto, o *ciclo de interrupção* é simples e previsível (Figura 12.8). O conteúdo atual de PC deve ser salvo para que o processador possa resumir a atividade normal depois da interrupção. Assim, os conteúdos de PC são transferidos para MBR para serem gravados na memória. A posição especial de memória reservada para este fim é carregada em MAR a partir da unidade de controle. Isso poderia ser, por exemplo, um ponteiro de pilha. O PC é preenchido com o endereço da rotina de interrupção. Como resultado disso, o próximo ciclo de instrução começará obtendo a instrução apropriada.

Figura 12.6 Fluxo de dados do ciclo de busca

MBR = registrador de buffer de memória
 MAR = registrador de endereço de memória
 IR = registrador da instrução
 PC = contador de programa

Figura 12.7 Fluxo de dados do ciclo indireto

12.4 Pipeline de instruções

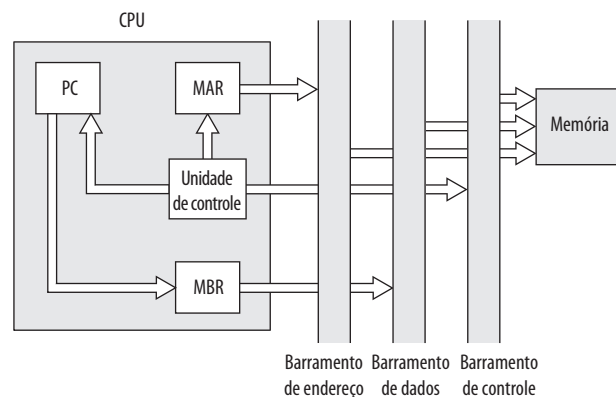
À medida que os sistemas computacionais evoluem, um melhor desempenho pode ser obtido tirando vantagens das melhorias na tecnologia, como por exemplo circuitos mais rápidos. Além disso, melhorias organizacionais no processador podem melhorar o desempenho. Nós já vimos alguns exemplos disso, como o uso de múltiplos registradores no lugar de um único acumulador e o uso de memória cache. Outra abordagem organizacional bastante comum é o pipeline da instrução.



Estratégia do pipeline

Pipeline de instrução é semelhante ao uso de uma linha de montagem numa planta industrial. Uma linha de montagem tira a vantagem do fato de que um produto passa por vários estágios da produção. Ao implantar o processo de produção em uma linha de montagem, produtos em vários estágios podem ser trabalhados simultaneamente. Este processo é também chamado de *pipelining*, porque assim como em uma tubulação, novas entradas são aceitas num lado antes que as entradas aceitas anteriormente apareçam como saídas do outro lado.

Para aplicar este conceito à execução da instrução, precisamos reconhecer o fato de que uma instrução possui vários estágios. A Figura 12.5, por exemplo, quebra o ciclo da instrução em 10 tarefas que ocorrem em sequência. Certamente deve existir alguma oportunidade para aplicar o conceito de pipeline.

Figura 12.8 Fluxo de dados do ciclo de interrupção

Como uma abordagem simplificada, considere dividir o processamento da instrução em dois estágios: ler instrução e executar instrução. Existem momentos durante a execução de uma instrução em que a memória principal não está sendo acessada. Esse tempo poderia ser usado para obter a próxima instrução paralelamente com a execução da instrução atual. A Figura 12.9a ilustra essa abordagem. O pipeline possui dois estágios independentes. O primeiro obtém a instrução e a coloca no buffer. Quando o segundo estágio está livre, o primeiro passa para ele a instrução do buffer. Enquanto o segundo estágio está executando a instrução, o primeiro estágio aproveita qualquer ciclo de memória não utilizado para obter a próxima instrução e colocá-la no buffer. Isso é chamado de busca antecipada (*prefetch*) ou **busca sobreposta**. Observe que esta abordagem, que envolve buffer da instrução, requer mais registradores. Em geral, pipeline requer registradores que guardem os dados entre os estágios.

Deve estar claro que este processo irá acelerar a execução da instrução. Se os estágios de leitura e execução forem de duração igual, o ciclo da instrução será reduzido pela metade. No entanto, se olharmos este pipeline mais de perto (Figura 12.9b), veremos que dobrar essa taxa de execução é pouco provável por dois motivos:

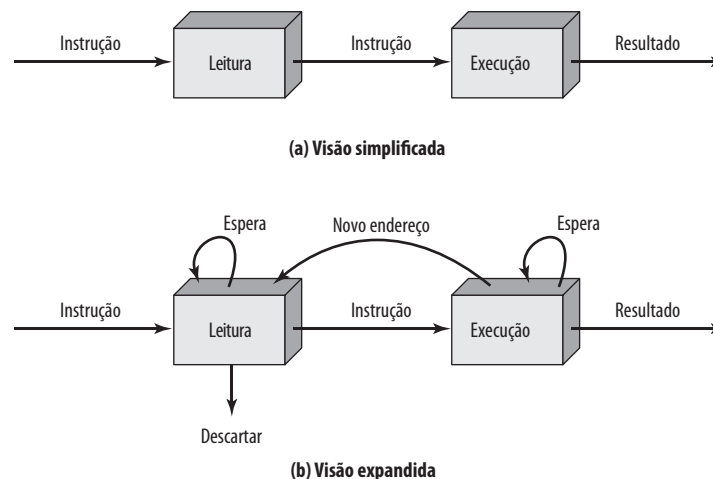
1. O tempo de execução normalmente será maior que o tempo de leitura. Execução envolve ler e armazenar operandos e o desempenho de alguma operação. Assim, o estágio de leitura pode ter que esperar por algum tempo antes de poder esvaziar o seu buffer.
2. Uma instrução de desvio condicional faz com que o endereço da próxima instrução a ser obtida não seja conhecido. Assim, o estágio de leitura deve esperar até que receba o endereço da próxima instrução do estágio de execução. O estágio de execução pode então ter que esperar até que a próxima instrução seja obtida.

Adivinhar pode reduzir o tempo perdido no segundo motivo. Uma regra simples é: quando uma instrução de desvio condicional passa do estágio de leitura para o de execução, o estágio de leitura obtém a próxima instrução na memória depois da instrução de desvio. Então, se o desvio não for tomado, nenhum tempo é perdido. Se o desvio for tomado, a instrução obtida deve ser descartada e uma nova instrução é lida.

Enquanto estes fatores reduzem a eficiência potencial de um pipeline de dois estágios, alguma aceleração ocorre. Para obter mais velocidade, o pipeline deve ter mais estágios. Vamos supor a seguinte decomposição do processamento da instrução:

- **Buscar instrução (FI, do inglês *Fetch Instruction*):** ler a próxima instrução esperada em um buffer.
- **Decodificar instrução (DI):** determinar o *opcode* e os especificadores dos operandos.
- **Calcular operandos (CO):** calcular o endereço efetivo de cada operando de origem. Isto pode envolver endereçamento por deslocamento, registrador indireto, indireto ou outras formas de cálculo de endereço.

Figura 12.9 Pipeline de instrução de dois estágios



- **Obter operandos (FO, do inglês *Fetch Operands*):** obter cada operando da memória. Operandos que estão nos registradores não precisam ser lidos da memória.
- **Executar instrução (EI):** efetuar a operação indicada e armazenar o resultado, se houver, na posição do operando de destino especificado.
- **Escrever operando (WO, do inglês *Write Operands*):** armazenar o resultado na memória.

Com esta decomposição, os vários estágios terão uma duração mais aproximada. Usando essa suposição, a Figura 12.10 mostra que um pipeline de seis estágios pode reduzir o tempo de execução de 9 instruções de 54 para 14 unidades de tempo.

Vários comentários devem ser levados em consideração: o diagrama supõe que cada instrução passa por todos os seis estágios do pipeline. Este nem sempre será o caso. Por exemplo, uma instrução de carga não precisa do estágio WO. No entanto, para simplificar o hardware do pipeline, o tempo é ajustado supondo-se que cada instrução requer todos os seis estágios. Além disso, o diagrama assume que todos os estágios podem ser executados em paralelo. Supõe-se também que não haverá conflitos de memória. Por exemplo, FI, FO e WO envolvem um acesso à memória. O diagrama implica que todos esses acessos podem ocorrer simultaneamente. A maioria de sistemas de memória não permitirá isso. No entanto, o valor desejado pode estar no cache, ou os estágios FO ou WO podem estar nulos. Assim, na maioria das vezes, os conflitos de memória não desacelerarão o pipeline.

Vários outros fatores limitam o aumento do desempenho. Se os seis estágios não forem de duração igual, haverá espera em vários estágios do pipeline, conforme discutido anteriormente para pipeline de dois estágios. Outra dificuldade é a instrução de desvio condicional, a qual pode invalidar várias leituras de instruções. Um evento imprevisível semelhante é a interrupção. A Figura 12.11 ilustra os efeitos do desvio condicional, usando o mesmo programa da Figura 12.10. Suponha que a instrução 3 seja um desvio condicional para instrução 15. Até que a instrução seja executada, não há nenhuma maneira de saber qual instrução virá a seguir. O pipeline, neste exemplo, simplesmente carrega a próxima instrução na sequência (instrução 4) e prossegue. Na Figura 12.10, o desvio não ocorre e obtivemos o benefício total do aumento do desempenho. Na Figura 12.11, o desvio ocorre. Isso não é determinado até o fim da unidade de tempo 7. Neste ponto, precisamos limpar o pipeline das instruções que não são úteis. Durante a unidade de tempo 8, a instrução 15 entra no pipeline. Nenhuma instrução é completada durante as unidades de tempo de 9 a 12; esta é uma penalidade de

Figura 12.10 Diagrama de tempo para operação do pipeline da instrução

	Tempo →													
	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Instrução 1	FI	DI	CO	FO	EI	WO								
Instrução 2		FI	DI	CO	FO	EI	WO							
Instrução 3			FI	DI	CO	FO	EI	WO						
Instrução 4				FI	DI	CO	FO	EI	WO					
Instrução 5					FI	DI	CO	FO	EI	WO				
Instrução 6						FI	DI	CO	FO	EI	WO			
Instrução 7							FI	DI	CO	FO	EI	WO		
Instrução 8								FI	DI	CO	FO	EI	WO	
Instrução 9									FI	DI	CO	FO	EI	WO

Figura 12.11 O efeito de um desvio condicional na operação do pipeline da instrução

	Tempo →							← Penalidade por desvio						
	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Instrução 1	FI	DI	CO	FO	EI	WO								
Instrução 2		FI	DI	CO	FO	EI	WO							
Instrução 3			FI	DI	CO	FO	EI	WO						
Instrução 4				FI	DI	CO	FO							
Instrução 5					FI	DI	CO							
Instrução 6						FI	DI							
Instrução 7							FI							
Instrução 15								FI	DI	CO	FO	EI	WO	
Instrução 16									FI	DI	CO	FO	EI	WO

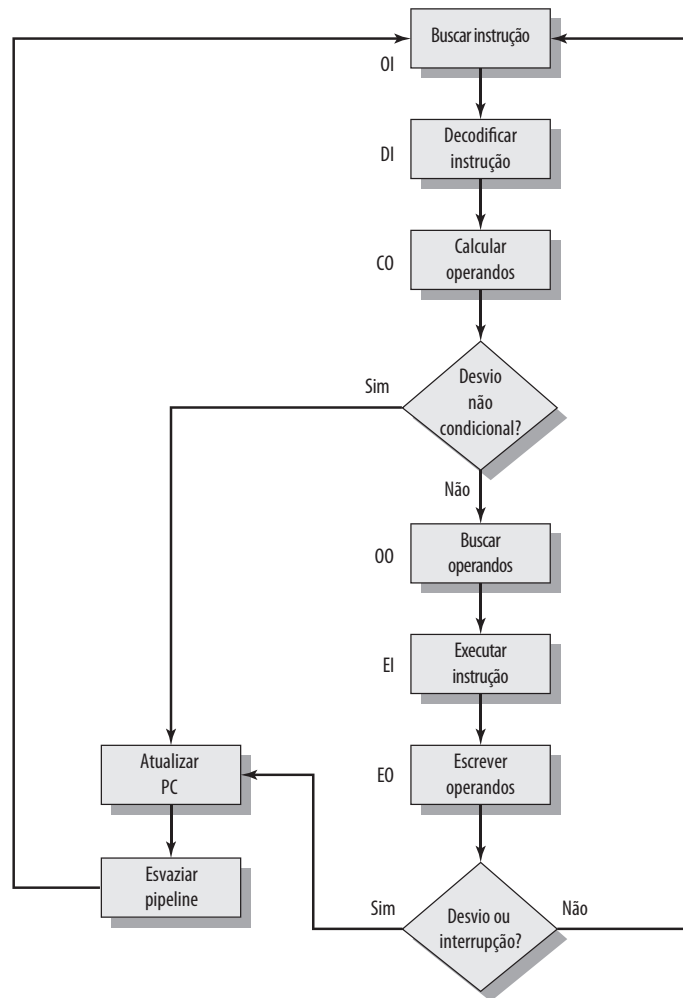
desempenho porque não pudemos antecipar o desvio. A Figura 12.12 indica a lógica necessária para pipeline computar desvios e interrupções.

Outro problema que não tinha aparecido na nossa organização simples de dois estágios surge agora. O estágio CO pode depender do conteúdo de um registrador que pode ser alterado por uma instrução anterior que ainda esteja no pipeline. Outros conflitos de registradores e memória desse tipo podem ocorrer. O sistema precisa ter uma lógica para lidar com esse tipo de conflitos.

Para esclarecer a operação do pipeline, pode ser útil olhar uma solução alternativa. As figuras 12.10 e 12.11 mostram o progresso de tempo horizontalmente através das figuras, onde cada linha mostra o progresso de uma instrução específica. A Figura 12.13 mostra a mesma sequência de eventos com o tempo sendo mostrado verticalmente e cada linha representando o estado do pipeline em um dado ponto no tempo. Na Figura 12.13a (que corresponde à Figura 12.10), o pipeline está cheio no tempo 6, com 6 instruções diferentes em vários estágios da execução e permanece cheio até o tempo 9; supondo que a instrução I9 seja a última a ser executada. Na Figura 12.13b (que corresponde à Figura 12.11), o pipeline está cheio nos tempos 6 e 7. No tempo 7, a instrução 3 está no estágio de execução e executa um desvio para instrução 15. Neste ponto, as instruções de I4 até I7 são retiradas do pipeline de tal forma que, no tempo 8, apenas duas instruções estão no pipeline, I3 e I15.

Da discussão anterior seria possível concluir que quanto maior o número de estágios no pipeline, maior será a taxa de execução. Alguns dos projetistas de IBM S/360 apontaram dois fatores que frustram este aparentemente simples padrão para projetos de alto desempenho (ANDERSON, SPARACIO e TOMASULO, 1967), ao passo que eles continuam sendo elementos que projetistas ainda precisam considerar:

1. Em cada estágio do pipeline, existe algum esforço extra envolvido para movimentação de dados de buffer para buffer e para efetuar várias funções de preparações e entregar de dados. Esse esforço extra pode desacelerar sensivelmente o tempo total de execução de uma única instrução. Isso é significativo quando instruções sequenciais são dependentes logicamente umas das outras, ou pelo uso pesado de desvios ou pelas dependências de acesso à memória.
2. A quantidade de lógica de controle necessária para lidar com dependências de memória e registradores e para otimizar o uso do pipeline aumenta imensamente com o número de estágios. Isso pode levar a uma situação onde a lógica que controla a passagem entre os estágios é mais complexa do que os estágios sendo controlados.

Figura 12.12 Pipeline de instrução de uma CPU de seis estágios

Outra consideração é o tempo de resposta: leva tempo para os buffers do pipeline operarem e isso aumenta o tempo do ciclo da instrução.

Pipeline de instrução é uma técnica poderosa para melhorar o desempenho, porém requer um projeto cuidadoso para alcançar ótimos resultados com uma complexidade razoável.



Desempenho do pipeline

Nesta subseção desenvolvemos algumas medições simples de desempenho do pipeline e a correspondente melhoria da velocidade (baseado em uma discussão em Hwang (1993)). O tempo de ciclo τ de uma instrução do pipeline é o tempo necessário para que a instrução avance um estágio dentro do pipeline; cada coluna nas figuras 12.10 e 12.11 representa um tempo de ciclo. O tempo de ciclo pode ser determinado como

$$\tau = \max[\tau_i] + d = \tau_m + d \quad 1 \leq i \leq k,$$

onde

τ_i = tempo de demora de resposta do circuito no estágio i do pipeline.

τ_m = tempo de demora máximo do estágio (demora do estágio que apresenta o maior tempo de demora de resposta).

k = número de estágios na instrução do pipeline.

d = tempo de resposta de um ciclo necessário para avançar sinais e dados de um estágio para o próximo.

Figura 12.13 Descrição alternativa de um pipeline

	FI	DI	CO	FO	EI	WO
1	I1					
2	I2	I1				
3	I3	I2	I1			
4	I4	I3	I2	I1		
5	I5	I4	I3	I2	I1	
6	I6	I5	I4	I3	I2	I1
7	I7	I6	I5	I4	I3	I2
8	I8	I7	I6	I5	I4	I3
9	I9	I8	I7	I6	I5	I4
10		I9	I8	I7	I6	I5
11			I9	I8	I7	I6
12				I9	I8	I7
13					I9	I8
14						I9

(a) Sem desvios

	FI	DI	CO	FO	EI	WO
1	I1					
2	I2	I1				
3	I3	I2	I1			
4	I4	I3	I2	I1		
5	I5	I4	I3	I2	I1	
6	I6	I5	I4	I3	I2	I1
7	I7	I6	I5	I4	I3	I2
8	I15					I3
9	I16	I15				
10		I16	I15			
11			I16	I15		
12				I16	I15	
13					I16	I15
14						I16

(b) Com desvios condicionais

Em geral, o tempo de resposta d é equivalente a um pulso de clock e $\tau_m \gg d$. Suponha agora que n instruções são processadas, sem desvios. Seja $T_{k,n}$ o tempo total necessário para que um pipeline com k estágios processe n instruções. Então

$$T_{k,n} = [k + (n - 1)]\tau \quad (12.1)$$

Um total de k ciclos é necessário para completar a execução da primeira instrução e o restante de $n - 1$ instruções requerem $n - 1$ ciclos.² Esta equação é facilmente verificada a partir da Figura 12.10. A nona instrução completa no ciclo de tempo 14:

$$14 = [6 + (9 - 1)]$$

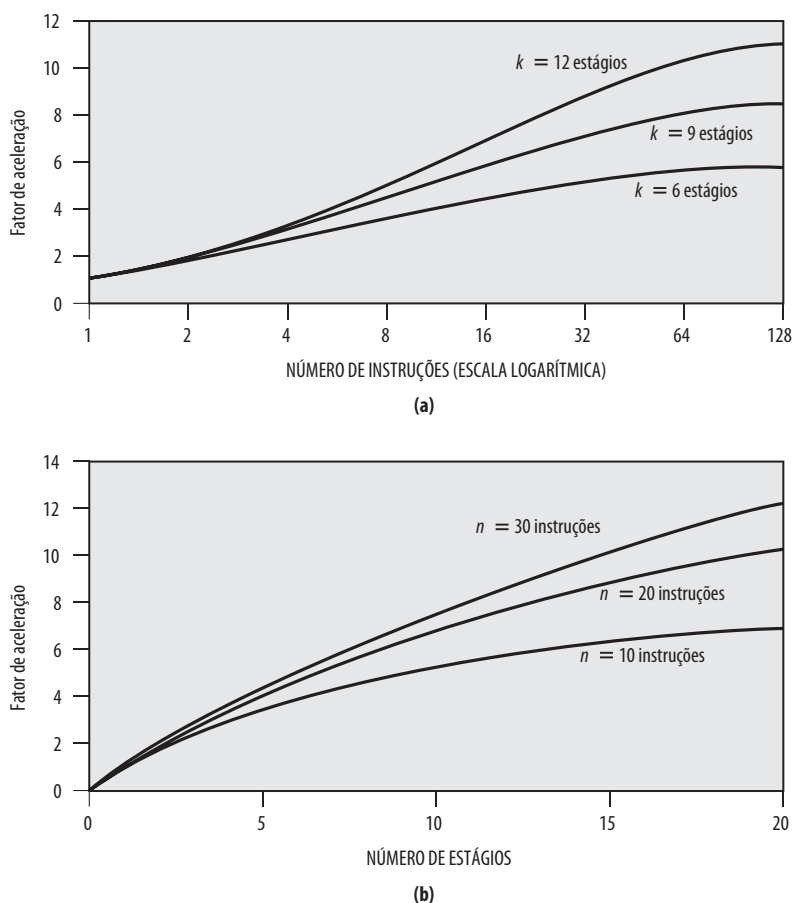
Considere agora um processador com funções equivalentes, mas sem pipeline, e suponha que o tempo do ciclo da instrução seja $k\tau$. O fator de aceleração para a instrução do pipeline comparado com a execução sem pipeline é definido como

$$S_k = \frac{T_{1,n}}{T_{k,n}} = \frac{nk\tau}{[k + (n - 1)]\tau} = \frac{nk}{k + (n - 1)} \quad (12.2)$$

A Figura 12.14a mostra o fator de aceleração como sendo uma função do número de instruções que são executadas sem um desvio. Conforme esperado, no limite ($n \rightarrow \infty$), temos uma aceleração pelo fator k . A Figura 12.14b mostra o fator de aceleração como uma função do número de estágios no pipeline da instrução.³ Neste caso, o fator de aceleração se aproxima do número de instruções que podem ser inseridas no pipeline

² Estamos sendo um pouco negligentes aqui. O tempo de ciclo apenas irá se igualar ao valor máximo de τ quando todos os estágios estiverem cheios. No começo, o tempo de ciclo pode ser menor para o primeiro ou alguns primeiros ciclos.

³ Observe que o eixo x é logarítmico na Figura 12.4a e linear na Figura 12.14b.

Figura 12.14 Fatores de aceleração com pipeline da instrução

sem desvios. Assim, quanto maior o número de estágios do pipeline, maior o potencial para aceleração. No entanto, por uma questão prática, os ganhos potenciais dos estágios adicionais do pipeline são confrontados pelo aumento do custo, demoras entre estágios e pelo fato de que os desvios irão requerer o esvaziamento do pipeline.



Hazards do pipeline

Na subseção anterior mencionamos algumas das situações que podem resultar em um desempenho de pipeline menor que a ótima. Nesta subseção examinaremos essa questão de uma forma mais sistemática. O Capítulo 14 retoma esta questão em mais detalhes, depois que tivermos introduzido as complexidades encontradas em organizações de pipelines superescalares.

Um **hazard de pipeline** ocorre quando o pipeline, ou alguma parte dele, precisa parar porque as condições não permitem a execução contínua. Tal parada do pipeline é também conhecida como *bolha de pipeline*. Existem três tipos de hazards: recurso, dados e controle.

HAZARDS DE RECURSOS Um hazard de recursos ocorre quando duas (ou mais) instruções que já estão no pipeline precisam do mesmo recurso. O resultado é que as instruções precisam ser executadas em série em vez de em paralelo para uma parte do pipeline. Um hazard de recursos às vezes é chamado de *hazard estrutural*.

Vamos ver um exemplo simples de um hazard de recursos. Suponha um pipeline simplificado de cinco estágios no qual cada estágio ocupa um ciclo de clock. A Figura 12.15a mostra o caso ideal onde uma nova instrução entra no pipeline a cada ciclo de clock. Suponha agora que a memória principal tenha uma única porta e que todas as leituras e

escritas de instruções e dados devam ser executadas uma por vez. Além disso, ignore a memória cache. Neste caso, uma leitura ou escrita do operando na memória não pode ser executada em paralelo com o processo de se ler uma instrução. Isso é ilustrado na Figura 12.15b, onde assumimos que o operando de origem para instrução I1 está na memória, em vez de em um registrador. Portanto, o estágio de busca da instrução deve ficar ocioso por um ciclo antes de começar a busca da instrução para instrução I3. A figura assume que todos os outros operandos estejam nos registradores.

Outro exemplo de um conflito de recurso é uma situação onde várias instruções estão prontas para entrar na fase de execução da instrução e existe apenas uma ALU. Uma solução para tal hazard de recursos é aumentar os recursos disponíveis, como ter múltiplas portas para memória principal ou múltiplas unidades de ALU.



Analizador de tabela de reservas

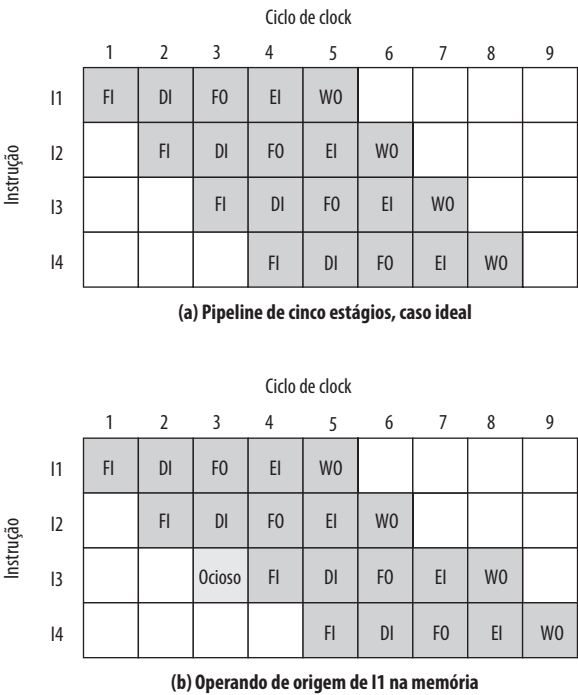
Uma abordagem para analisar conflitos de recursos e ajudar no projeto de pipelines é a tabela de reservas. Nós examinamos as tabelas de reservas no Apêndice I.

HAZARDS DE DADOS Um hazard de dados ocorre quando há um conflito no acesso de uma posição de operando. De um modo geral, podemos definir o hazard da seguinte forma: duas instruções em um programa estão para ser executadas na sequência e ambas acessam um determinado operando de memória ou registrador. No entanto, se as instruções são executadas em um pipeline, então é possível que o valor do operando seja atualizado de tal forma que produza um resultado diferente do que seria com uma execução estritamente sequencial. Em outras palavras, o programa produz um resultado incorreto por causa do uso do pipelining.

Como um exemplo, considere a seguinte sequência de instrução de máquina para x86:

```
ADD EAX, EBX    /* EAX = EAX + EBX
SUB ECX, EAX    /* ECX = ECX - EAX
```

Figura 12.15 Exemplo de hazard de recursos



A primeira instrução soma o conteúdo dos registradores de 32 bits EAX e EBX e armazena o resultado em EAX. A segunda instrução subtrai o conteúdo de EAX de ECX e armazena o resultado em ECX. A Figura 12.16 mostra o comportamento do pipeline. A instrução ADD não atualiza EAX até o fim do estágio 5, o qual ocorre no ciclo 5 de clock, mas a instrução SUB precisa desse valor no começo do seu estágio 2, o qual ocorre no ciclo 4 de clock. Para manter a operação correta, o pipeline deve atrasar por dois ciclos de clock. Assim, na falta de hardware especial e de algoritmos específicos para evitar isso, o hazard de dados pode resultar no uso ineficiente do pipeline.

Existem três tipos de hazards de dados:

- **Leitura após escrita ou dependência verdadeira:** uma instrução modifica um registrador ou uma posição de memória e uma instrução subsequente lê os dados dessa posição de memória ou registrador. O hazard ocorre quando a operação de leitura acontece antes da escrita ter sido completada.
- **Escrita após leitura ou antidependência:** uma instrução lê um registrador ou uma posição de memória e uma instrução subsequente escreve nessa posição. O hazard ocorre se a operação de escrita é completada antes da operação de leitura.
- **Escrita após escrita ou dependência de saída:** duas instruções escrevem na mesma posição. O perigo ocorre se as operações de escrita acontecerem na sequência inversa da esperada.

O exemplo da Figura 12.16 é um hazard de dados do tipo verdadeira dependência. Outros dois hazards são entendidos melhor no contexto de organizações superescalares, discutidas no Capítulo 14.

HAZARDS DE CONTROLE Um hazard de controle, também conhecido como *hazard de desvio*, acontece quando o pipeline toma decisão errada ao prever um desvio e assim acaba trazendo instruções dentro do pipeline que precisam ser descartadas logo em seguida. Discutimos abordagens para lidar com hazards de controle a seguir.



Lidando com desvios

Um dos principais problemas ao se projetar um pipeline de instruções é garantir um fluxo estável de instruções para os estágios iniciais do pipeline. O primeiro impedimento, conforme já vimos, é a instrução condicional de desvio condicional. Até que a instrução seja executada de fato, é impossível dizer se o desvio será tomado ou não.

Uma série de abordagens foram implementadas para lidar com desvios condicionais:

- Múltiplos fluxos.
- Busca antecipada do alvo do desvio.
- Buffer de laço de repetição.
- Previsão de desvio (*branch prediction*).
- Desvio atrasado.

MÚLTIPLOS FLUXOS Um pipeline simples tem penalidades na execução de uma instrução de desvio, pois precisa buscar duas instruções e pode fazer a escolha errada. Uma abordagem tipo força bruta é replicar as partes iniciais do pipeline e permitir que pipeline obtenha as duas instruções, fazendo assim uso de dois fluxos. Existem dois problemas com esta abordagem:

Figura 12.16 Exemplo de perigo de dados

		Ciclo de clock									
		1	2	3	4	5	6	7	8	9	10
ADD EAX, EBX		FI	DI	FO	EI	EO					
SUB ECX, EAX			FI	DI	Ocioso		FO	EI	WO		
I3				FI			DI	FO	EI	WO	
I4							FI	DI	FO	EI	WO

- Com múltiplos pipelines, existem atrasos no acesso aos registradores e à memória.
- Instruções de desvio adicionais podem entrar no pipeline (ou em qualquer dos fluxos) antes que a decisão de desvio original seja resolvida. Cada uma dessas instruções precisa de um fluxo adicional.

Apesar dessas desvantagens, esta estratégia pode melhorar o desempenho. Exemplos de máquinas com dois ou mais fluxos de pipeline são IBM 370/168 e IBM 3033.

BUSCA ANTECIPADA DO ALVO DO DESVIO Quando um desvio condicional é reconhecido, o alvo do desvio é lido antecipadamente, além da instrução que segue o desvio. Esse alvo é então salvo até que a instrução de desvio seja executada. Se o desvio for tomado, o alvo já foi obtido.

IBM 360/91 utiliza esta abordagem.

BUFFER DE LAÇO DE REPETIÇÃO Um buffer de laço de repetição é uma memória pequena e extremamente rápida mantida pelo estágio do pipeline de busca da instrução e que contém n instruções mais recentemente lidas na sequência. Se um desvio está para ser tomado, o hardware primeiro verifica se o alvo do desvio já está no buffer. Se estiver, a próxima instrução é obtida do buffer. O buffer de laço de repetição possui três benefícios:

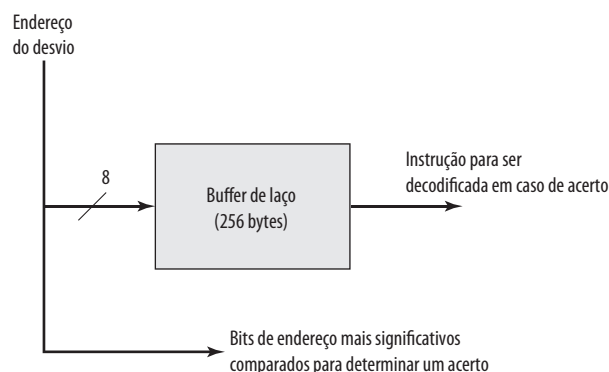
1. Com o uso de busca antecipada, o buffer de laço conterá algumas instruções em sequência na frente do endereço da instrução atual. Assim, as instruções obtidas na sequência estarão disponíveis sem o tempo usual de acesso à memória.
2. Se um desvio para um alvo estiver apenas algumas posições à frente do endereço da instrução de desvio, o alvo já estará no buffer. Isso é útil para ocorrências muito comuns das sequências *IF-THEN* e *IF-THEN-ELSE*.
3. Esta estratégia é particularmente bem adaptada para lidar com laços ou iterações; por isso o nome de *buffer de laço de repetição*. Se o buffer de laço de repetição for suficientemente grande para conter todas as instruções de um laço, então essas instruções precisam ser obtidas da memória apenas uma vez, na primeira iteração. Para iterações subsequentes, todas as instruções necessárias já estão no buffer.

O buffer de laço é semelhante em princípio a um cache dedicado para instruções. A diferença é que buffer de laço guarda apenas instruções na sequência e tem um tamanho menor, tendo assim um custo menor também.

A Figura 12.17 mostra um exemplo de buffer de laço. Se buffer contém 256 bytes, e endereçamento de byte é usado, então os oito bits menos significativos são usados para indexar o buffer. Os bits mais significativos restantes são verificados para determinar se o alvo do desvio se encontra dentro do ambiente capturado pelo buffer.

Entre as máquinas que usam o buffer de laço de repetição estão algumas máquinas CDC (Star-100, 6600, 7600) e CRAY-1. Uma forma especial de buffer de laço de repetição está disponível no Motorola 68010 para executar um laço de três instruções envolvendo uma instrução DBcc (decremento e desvio na condição) (veja o Problema 12.14). Um buffer de três palavras é mantido e o processador executa estas instruções repetidamente até que a condição do laço seja satisfeita.

Figura 12.17 Buffer de laço de repetição





Simulador de previsão de desvio Buffer de alvo de desvio

PREVISÃO DE DESVIO Várias técnicas podem ser usadas para prever se um desvio será tomado. Entre as mais comuns estão as seguintes:

- Previsão nunca tomada.
- Previsão sempre tomada.
- Previsão por *opcode*.
- Chave tomada/não tomada.
- Tabela de histórico de desvio.

As três primeiras abordagens são estáticas: elas não dependem do histórico da execução até o momento da instrução do desvio condicional. As duas últimas são dinâmicas: elas dependem do histórico da execução.

As duas primeiras abordagens são mais simples. Ou elas assumem que o desvio nunca será tomado e continuam obtendo as instruções na sequência ou elas sempre assumem que o desvio será tomado e sempre obtêm o alvo do desvio. A abordagem de previsão nunca tomada é a mais popular de todos os métodos de previsão de desvios.

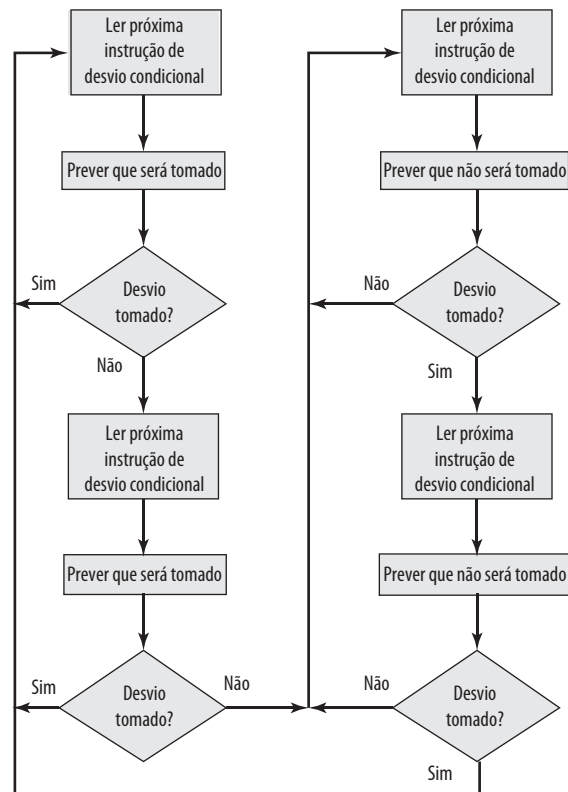
Estudos que analisaram o comportamento dos programas mostraram que os desvios condicionais são tomados em mais que 50% das vezes (Lilja, 1988⁶), então se o custo de busca antecipada dos dois caminhos é o mesmo, fazer a busca antecipada sempre do endereço do alvo do desvio deveria oferecer um desempenho melhor do que sempre fazer busca antecipada do caminho sequencial. No entanto, em uma máquina com paginação, fazer busca antecipada do alvo do desvio terá maior probabilidade de causar uma falha de página do que fazer busca antecipada da próxima instrução na sequência e, por isso, esta penalidade de desempenho deve ser levada em conta. Um mecanismo para evitar isso poderia ser empregado para reduzir tal penalidade.

A abordagem estática final toma a decisão com base no *opcode* da instrução de desvio. O processador assume que o desvio será feito para determinados *opcodes* de desvio e não para outros. Lilja (1988⁶) reporta taxas de sucesso superiores a 75% com esta estratégia.

As estratégias dinâmicas tentam melhorar a precisão da previsão armazenando um histórico de instruções de desvios condicionais de um programa. Por exemplo, um ou mais bits podem ser associados com cada instrução de desvio condicional que reflete o histórico recente da instrução. Estes bits são conhecidos como uma chave tomada/não tomada que direciona o processador a tomar uma determinada decisão na próxima vez que a instrução for encontrada. Normalmente esses bits de histórico não são associados com a instrução na memória principal. Em vez disso, eles são guardados em um armazenamento temporário de alta velocidade. Uma possibilidade é associar esses bits com qualquer instrução de desvio condicional que esteja na cache. Quando a instrução é substituída no cache, o seu histórico é perdido. Outra possibilidade é manter uma pequena tabela para instruções de desvio recentemente executadas com um ou mais bits de históricos para cada entrada. O processador poderia acessar a tabela de forma associativa, com uma cache, ou usando os bits de ordem mais baixa do endereço da instrução de desvio.

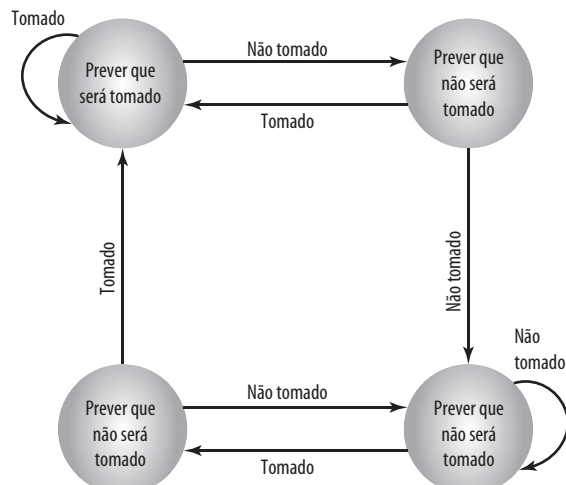
Com um bit único, tudo o que pode ser guardado é se a última execução dessa instrução resultou em um desvio ou não. Uma desvantagem de usar um bit único aparece no caso de uma instrução de desvio condicional que é quase sempre tomada como uma instrução de laço repetitiva. Com apenas um bit de histórico, um erro de previsão ocorrerá duas vezes para cada uso do laço: uma vez ao entrar no laço e outra vez ao sair.

Se dois bits são usados, eles podem ser utilizados para guardar o resultado das duas últimas instâncias da execução da instrução associada ou para guardar o estado de alguma outra forma. A Figura 12.18 mostra uma abordagem típica (veja o Problema 12.13 para outras possibilidades). Suponha que o algoritmo começa no canto superior esquerdo do fluxograma. À medida que cada instrução de desvio condicional subsequente encontrada é tomada, o processo de decisão prevê que o próximo desvio será tomado. Se uma única previsão for errada, o algoritmo continua prevendo que o próximo desvio será tomado. Apenas se dois desvios seguidos não forem tomados fará com que o algoritmo mude para o lado direito do fluxograma. Subsequentemente, o algoritmo irá prever que desvios não são tomados até que dois desvios em uma linha sejam tomados. Assim, o algoritmo requer duas previsões erradas em seguida para mudar a decisão da previsão.

Figura 12.18 Fluxograma de previsão de desvio

O processo de decisão pode ser representado de maneira mais compacta por uma máquina de estados finitos, mostrada na Figura 12.19. A representação de uma máquina de estados finitos é comumente usada na literatura.

O uso de bits de histórico, conforme descrito agora, tem uma desvantagem: se for decidido tomar o desvio, a instrução alvo não pode ser obtida até que o endereço do alvo, que é um operando dentro da instrução de desvio

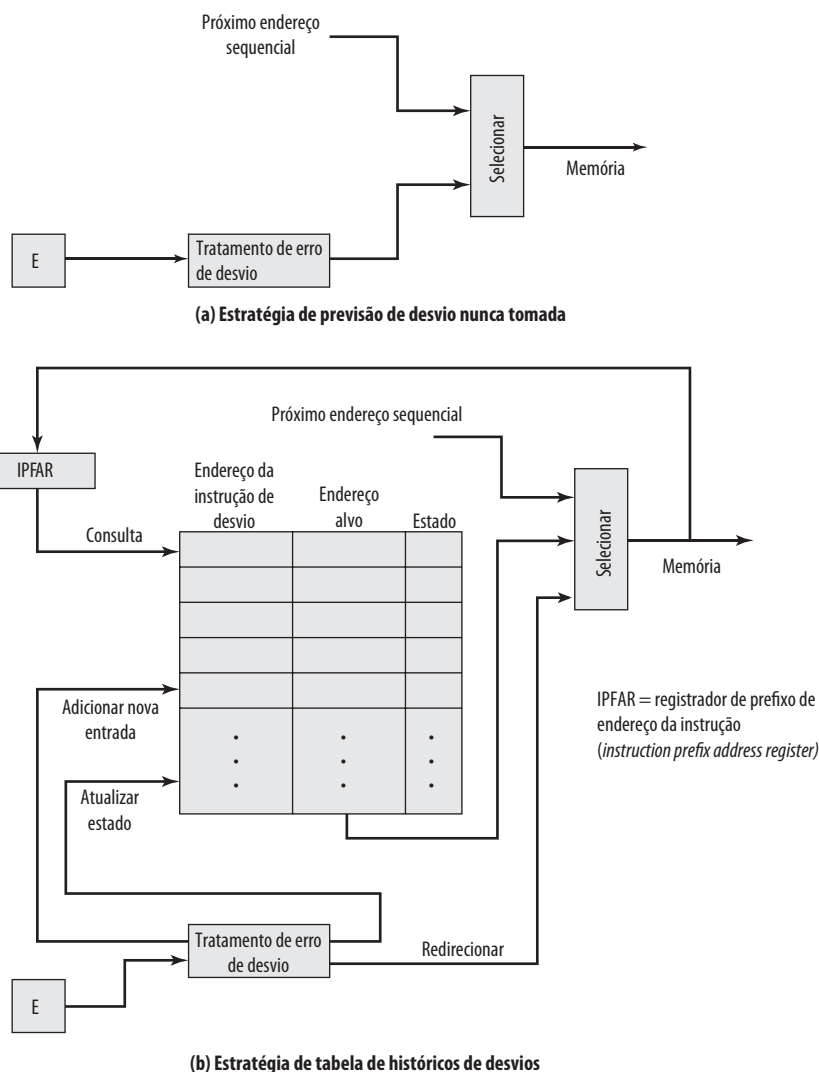
Figura 12.19 Diagrama de estados de previsão de desvio

condicional, seja decodificada. Uma eficiência maior pode ser alcançada se a leitura da instrução puder ser iniciada assim que a decisão de tomada de desvio for feita. Para este propósito, mais informações precisam ser salvas no que é conhecido como buffer de alvo do desvio ou tabela de histórico de desvios.

A tabela de histórico de desvios é uma pequena memória cache associada com o estágio de leitura da instrução do pipeline. Cada entrada da tabela consiste de três elementos: o endereço da instrução de desvio, algum número de bits de histórico que guardam o estado de uso dessa instrução e informação sobre a instrução alvo. Na maioria das propostas e implementações, esse terceiro campo contém o endereço da instrução alvo. Outra possibilidade é que o terceiro campo contenha a instrução alvo em si. A negociação é clara: armazenar o endereço do alvo necessita de uma tabela menor, porém um tempo maior para obter a instrução se comparado com armazenar a instrução alvo (RECHES e WEISS, 1998).

A Figura 12.20 contraria esse esquema com uma estratégia de prever que nunca será tomada. Com essa estratégia, o estágio de leitura da instrução sempre obtém o próximo endereço na sequência. Se um desvio for tomado, alguma lógica no processador detecta isso e instrui que a próxima instrução seja obtida do endereço alvo (além de esvaziar o pipeline). A tabela de histórico de desvio é tratada como uma cache. Cada busca antecipada dispara uma busca na tabela de histórico de desvios. Se nenhuma ocorrência correspondente for encontrada, o próximo endereço sequencial é usado para leitura. Se uma ocorrência correspondente for encontrada, uma previsão é feita com base no estado da instrução: ou o próximo endereço sequencial ou o endereço do alvo do desvio é informado para lógica selecionada.

Figura 12.20 Lidando com desvios



Quando uma instrução de desvio é executada, o estágio de execução sinaliza a lógica da tabela de histórico de desvios com o resultado. O estado da instrução é atualizado para refletir uma previsão correta ou incorreta. Se a previsão for incorreta, a lógica de seleção é redirecionada para o endereço correto para próxima leitura. Quando uma instrução de desvio condicional que não esteja na tabela é encontrada, ela é adicionada à tabela e uma das entradas existentes é descartada com uso de um dos algoritmos de substituição de cache discutidos no Capítulo 4.

Um refinamento da abordagem do histórico de desvios é referenciado como histórico de desvios de dois níveis ou baseado em correlação (YEH e PATT, 1991^m). Esta abordagem é baseada na suposição de que, apesar de nos desvios de laço o histórico de uma determinada instrução de desvio ser uma boa forma de previsão de comportamentos futuros, com estruturas de controle de fluxo mais complexas, a direção de um desvio é frequentemente correlacionada com a direção de desvios relacionados. Um exemplo disso é uma estrutura de *if-then-else* ou *case*. Existem diversas estratégias possíveis. Normalmente, o histórico global de desvios recentes (ou seja, o histórico de desvios mais recentes e não apenas desta instrução de desvio) é usado junto com o histórico da instrução de desvio atual. A estrutura geral é definida como uma correlação (m, n) que usa o comportamento dos últimos m desvios para escolher uma previsão de desvio a partir de 2^m n -bits para a atual instrução de desvio. Em outras palavras, um histórico de n -bits é guardado para um dado desvio para cada combinação possível de desvios tomada por m desvios mais recentes.

DESVIO ATRASADO É possível melhorar o desempenho do pipeline rearranjando automaticamente as instruções dentro de um programa, para que as instruções ocorram depois do que realmente desejado. Esta abordagem intrigante é examinada no Capítulo 13.



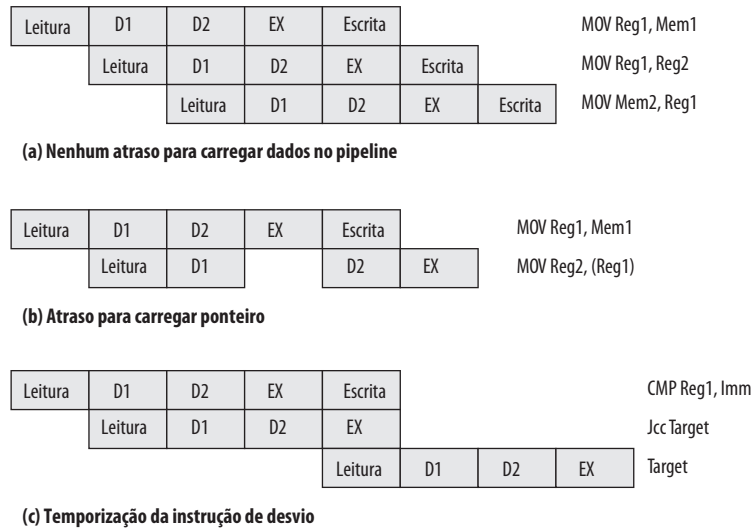
Pipeline de Intel 80486

Um exemplo instrutivo de um pipeline de instruções é o de Intel 80486. Ele implementa um pipeline de cinco estágios:

- **Leitura:** instruções são obtidas a partir da cache ou da memória externa e são colocadas em um de dois buffers de busca antecipada de 16 bits. O objetivo do estágio de leitura é preencher os buffers de busca antecipada com dados novos assim que os dados antigos tenham sido consumidos pelo decodificador da instrução. Como as instruções têm tamanhos variáveis (de 1 a 11 bytes sem contar prefixos), o estado do estágio da busca antecipada em relação a outros estágios varia de instrução para instrução. Em média, em torno de cinco instruções são obtidas com cada carga de 16 bytes (CRAWFORD, 1990ⁿ). O estágio de leitura opera independentemente de outros estágios para manter os buffers de busca antecipada cheios.
- **Estágio de decodificação 1:** toda a informação de *opcode* e modo de endereçamento é decodificada no estágio D1. A informação requerida, assim como a informação sobre o tamanho da instrução, é incluída em, no máximo, nos 3 primeiros bytes da instrução. Por isso, os 3 bytes são passados para o estágio D1 a partir dos buffers de busca antecipada. O decodificador D1 pode então direcionar o estágio D2 para pegar o restante da instrução (dados imediatos e de deslocamento), a qual não está envolvida na decodificação em D1.
- **Estágio de decodificação 2:** o estágio D2 traduz cada *opcode* em sinais de controle para ALU. Ele também controla o cálculo de modos de endereçamento mais complexos.
- **Execução:** este estágio inclui operações de ALU, acesso a cache e atualização de registradores.
- **Escrita:** este estágio, se necessário, atualiza registradores e flags de estado modificados durante o processo da execução anterior. Se a instrução corrente atualiza a memória, o valor computado é enviado para a cache e a interface de barramento escreve nos buffers ao mesmo tempo.

Com uso de dois estágios de decodificação, o pipeline pode sustentar um fluxo de quase uma instrução por ciclo de clock. Instruções complexas e desvios condicionais podem diminuir essa taxa.

A Figura 12.21 mostra exemplos da operação do pipeline. A parte (a) mostra que não há atraso introduzido no pipeline quando um acesso à memória é necessário. No entanto, conforme a parte (b) mostra, pode haver um atraso para valores usados para calcular endereço de memória. Isto é, se um valor é carregado da memória em um registrador e esse registrador é então usado como um registrador base na próxima instrução, o processador irá atrasar por um ciclo. Neste exemplo, o processador acessa a cache no estágio EX da primeira execução e armazena o valor obtido no registrador durante o estágio Escrita. No entanto, a próxima instrução precisa desse registrador no seu estágio D2. Quando o estágio D2 se alinha com o estágio Escrita da instrução anterior, caminhos de passa-

Figura 12.21 Exemplos de pipeline da instrução do 80486

gem de sinal permitem que o estágio D2 tenha acesso aos mesmos dados usados pelo estágio Escrita para escrita, economizando um estágio do pipeline.

A Figura 12.21c ilustra a temporização de uma instrução de desvio, supondo que desvio seja tomado. A instrução de comparação atualiza os códigos condicionais no estágio Escrita e os caminhos de passagem tornam isso disponível para o estágio EX da instrução de salto ao mesmo tempo. Em paralelo, o processador executa o ciclo de busca especulativo para o alvo do salto durante o estágio EX da instrução de salto. Se o processador determinar uma condição de desvio falsa, ele descarta a busca antecipada e continua a execução com a próxima instrução da sequência (já lida e decodificada).

12.5 Família de processadores x86

A organização x86 evoluiu consideravelmente ao longo dos anos. Nesta seção analisamos alguns dos detalhes das mais recentes organizações dos processadores, focando em elementos comuns em processadores únicos. O Capítulo 14 fala sobre aspectos superescalares de x86 e o Capítulo 18 analisa organização de múltiplos núcleos. Uma visão da organização do processador Pentium 4 é mostrada na Figura 4.18.

Organização dos registradores

A organização dos registradores inclui os seguintes tipos de registradores (Tabela 12.2):

- **Propósito geral:** existem oito registradores de 32 bits de uso geral (veja Figura 12.3c). Eles podem ser usados para todos os tipos de instruções x86 e também podem guardar operandos para cálculo de endereços. Além disso, alguns desses registradores servem também para propósitos específicos. Por exemplo, as instruções de *string* usam o conteúdo dos registradores ECX, ESI e EDI como operandos sem precisar referenciar esses registros explicitamente na instrução. Como resultado, uma série de instruções pode ser codificada de forma mais compactada. No modo 64 bits, existem 16 registradores de propósito geral de 64 bits.
- **Segmento:** seis registradores de segmento de 16 bits contêm seletores de segmento, os quais indexam as tabelas de segmentos, conforme discutido no Capítulo 8. O registrador de segmento de código (CS) referencia o segmento contendo as instruções a serem executadas. O registrador de segmento de pilha (SS) referencia o segmento contendo uma pilha visível ao usuário. Os registradores de segmento restantes (DS, ES, FS e GS) possibilitam ao usuário referenciar até quatro segmentos de dados separados ao mesmo tempo.

Tabela 12.2 Registradores do processador x86**(a) Unidade de inteiros no modo 32-bits**

Tipo	Número	Tamanho (bits)	Propósito
Propósito geral	8	32	Registradores de propósito geral do usuário
Segmento	6	16	Contém seletores de segmento
EFLAGS	1	32	Bits de estado e controle
Ponteiro da instrução	1	32	Ponteiro da instrução

(b) Unidade de inteiros no modo 64-bits

Tipo	Número	Tamanho (bits)	Propósito
Propósito geral	16	32	Registradores de uso geral do usuário
Segmento	6	16	Contém seletores de segmento
RFLAGS	1	64	Bits de estado e controle
Ponteiro da instrução	1	64	Ponteiro da instrução

(c) Unidade de ponto flutuante

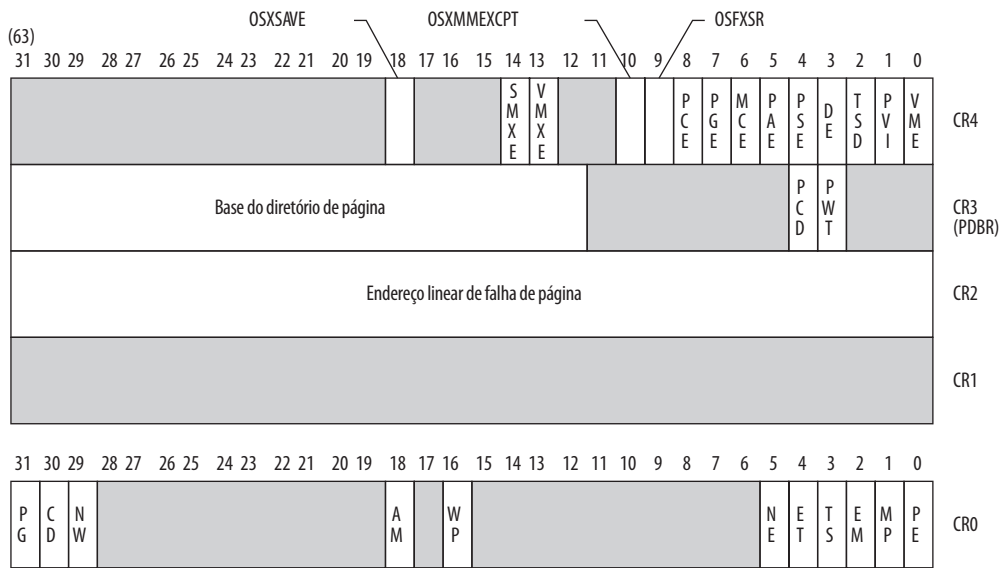
Tipo	Número	Tamanho (bits)	Propósito
Númerico	8	80	Armazena números de ponto flutuante
Controle	1	16	Bits de controle
Estado	1	16	Bits de estado
Palavra de marcação	1	16	Especifica o conteúdo de registradores numéricos
Ponteiro da instrução	1	48	Apona para instrução interrompida pela execução
Ponteiro de dados	1	48	Apona para operando interrompido pela exceção

- **Flags:** os registradores EFLAGS de 32 bits contêm códigos condicionais e vários bits de modo. No modo 64 bits, o registrador é estendido para 64 bits e é referenciado como RFLAGS. Na atual definição da arquitetura, os 32 bits superiores de RFLAGS não são usados.
- **Ponteiro da instrução:** contém o endereço da instrução corrente.

Existem também registradores dedicados especialmente para unidade de ponto flutuante:

- **Númerico:** cada registrador guarda um número de ponto flutuante de 80 bits de precisão estendida. Existem 8 registradores que funcionam como uma pilha, com operações push e pop disponíveis no conjunto de instruções.
- **Controle:** o registrador de controle de 16 bits contém bits que controlam a operação da unidade de ponto flutuante, incluindo o tipo de arredondamento; precisão única, dupla ou estendida; e bits para habilitar ou desabilitar diversas condições de exceção.
- **Estado:** o registrador de estado de 16 bits contém bits que refletem o atual estado da unidade de ponto flutuante, incluindo um ponteiro de 3 bits para o topo da pilha; códigos condicionais que reportam a saída da última operação; e flags de exceção.
- **Palavra de marcação:** este registrador de 16 bits contém uma marca de 2 bits para cada registrador numérico de ponto flutuante, que indica a natureza do conteúdo do registrador correspondente. Quatro valores possíveis são válidos, zero, especial (NaN, infinito, desnormalizado e vazio). Esses marcadores possibilitam que os programas verifiquem o conteúdo de um registrador numérico sem executar decodificação complexa dos dados atuais no registrador. Por exemplo, quando há uma troca de contexto, o processador não precisa salvar nenhum registrador de ponto flutuante que esteja vazio.

ID	=	flag de identificação	DF	=	flag direcional
VIP	=	Interrupção virtual pendente	IF	=	flag para habilitar interrupção
VIF	=	flag de interrupção virtual	TF	=	flag de <i>trap</i>
AC	=	Verificação de alinhamento	SF	=	flag de <i>signal</i>
VM	=	8086 modo virtual	ZF	=	flag Zero
RF	=	flag de resumo	AF	=	flag de <i>carry</i> auxiliar
NT	=	flag tarefa aninhada	PF	=	flag de paridade
IOPL	=	de privilégio de EIS	CF	=	flag de <i>carry</i>
OF	=	flag de <i>overflow</i>			

Figura 12.23 Registradores de controle de x86

Áreas sombreadas indicam bits reservados.

OSXSAVE = habilita bit XSAVE

SMXE = habilita extensões do modo de segurança

VMXE = habilita extensões de máquina virtual

OSXMMEXCPT = Suporta excessões SIMD FP não mascaradas

OSFXSR = Suporta FXSAVE, FXSTOR

PCE = habilita conector de desempenho

PGE = habilita paginação global

MCE = habilita verificação de máquina

PAE = extensão de endereço físico

PSE = extensões de tamanho de página

DE = extensões de depuração

TSD = desabilitar *time stamp*

PVI = interrupções virtuais no modo protegido

VME = modo de extensão virtual de 8086

PCD = desabilita cache de página

PWT = escrita transparente em nível de página

PG = paginação

CD = desabilita cache

NW = not write through

AM = máscara de alinhamento

WP = proteção de escrita

NE = erro numérico

ET = tipo de extensão

TS = troca de tarefa

EM = emulação

MP = monitor do coprocessador

PE = habilitação de proteção

- **Emulação (EM):** definido quando o processador não possui uma unidade de ponto flutuante e causa uma interrupção quando uma tentativa é feita para execução de instruções de ponto flutuante.
- **Troca de tarefa (TS, do inglês *task switched*):** indica que o processador trocou tarefas.
- **Tipo de extensão (ET, do inglês *extension type*):** não é usado em Pentium e máquinas posteriores; usado para indicar suporte para instruções de coprocessador matemático em máquinas anteriores.
- **Erro numérico (NE, do inglês *numeric error*):** habilita o mecanismo padrão para reportar erros de ponto flutuante em linhas de barramento externo.
- **Proteção de escrita (WP, do inglês *write protect*):** quando este bit é igual, páginas a zero de usuário com permissão de somente leitura podem ser escritas por um processo supervisor. Este recurso é útil para suportar criação de processos em alguns sistemas operacionais.
- **Máscara de alinhamento (AM, do inglês *alignment mask*):** habilita/desabilita verificação de alinhamento.
- **Not write through (NW):** seleciona o modo de operação de cache de dados. Quando o bit é igual a um cache de dados é inibido para atualizações de cache *write through*.
- **Desabilitar cache (CD — *cache disable*):** habilita/desabilita o mecanismo interno de preenchimento de cache.
- **Paginação (PG):** habilita/desabilita paginação.

Quando paginação está habilitada, os registradores CR2 e CR3 são válidos. O registrador CR2 guarda o endereço linear de 32 bits da última página acessada antes de uma interrupção de falha de página. Os 20 bits da extrema esquerda de CR3 guardam os 20 bits mais significativos do endereço base do diretório de página; o restante do endereço contém zeros. Dois bits de CR3 são usados como acionadores que controlam a operação de uma cache externa. Desabilitar a cache em nível de página (PCD, do inglês *page-level cache disable*) habilita ou desabilita a cache externa e o bit de escrita transparente em nível de página (PWT, do inglês *page-level writer transparent*) controla a escrita na cache externa.

Nove bits de controle adicionais são definidos em CR4:

- **Modo de extensão virtual de 8086 (VME, do inglês *virtual 8086 mode extensions*):** habilita o suporte para flag de interrupção virtual no modo virtual 8086.
- **Interrupções virtuais no modo protegido (PVI, do inglês *protected mode virtual interrupt*):** habilita o suporte para flag de interrupção virtual em modo protegido.
- **Desabilitar time stamp (TSD, do inglês *time stamp disable*):** desabilita a leitura a partir da instrução contadora de time stamp (RDTSC), a qual é usado para propósitos de depuração.
- **Extensões de depuração (DE, do inglês *debug extensions*):** habilita *breakpoints* de E/S; isto permite que o processador interrompa leituras e gravações de E/S.
- **Extensões de tamanho da página (PSE, do inglês *page size extensions*):** quando igual a um, habilita tamanhos de página grandes (páginas de 2 ou 4 MB); quando igual a zero, restringe páginas a 4 KB.
- **Extensão do endereço físico (PAE, do inglês *physical address extension*):** habilita linhas de endereço de A35 até A32 sempre que um novo modo especial de endereçamento, controlado por PSE, é habilitado.
- **Habilita verificação de máquina (MCE, do inglês *machine check enable*):** habilita interrupção de verificação de máquina, a qual ocorre quando um erro de paridade de dados acontece durante o ciclo de leitura de barramento ou quando um ciclo de barramento não é completado com sucesso.
- **Habilitar paginação global (PGE, do inglês *page global enable*):** habilita o uso de páginas globais. Quando PGE = 1 e uma troca de tarefa é efetuada, todas as entradas de TLB são excluídas com exceção daquelas marcadas como globais.
- **Habilita contador de desempenho (PCE, do inglês *performance counter enable*):** habilita a execução da instrução RDPMC (leitura de contador de desempenho) em qualquer nível de privilégio. Dois contadores de desempenho são usados para medir a duração de um tipo de evento específico e o número de ocorrências e um tipo de evento específico.

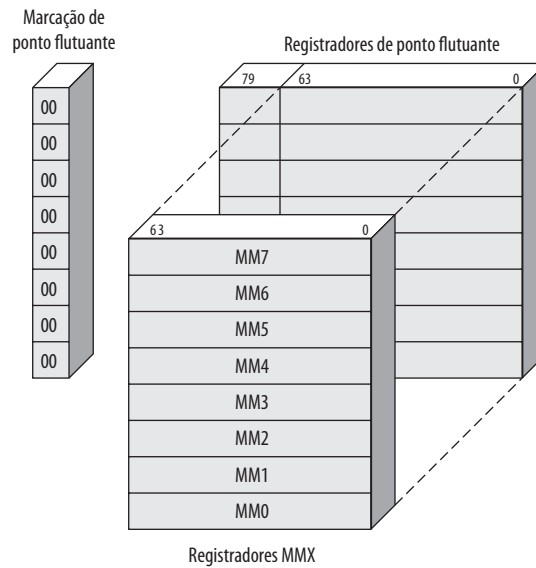
REGISTRADORES MMX Lembre da Seção 10.3 em que o x86 MMX usa vários tipos de dados de 64 bits. As instruções MMX fazem uso de campos de endereço de registrador de 3 bits para que oito registradores MMX sejam suportados. Na verdade, o processador não inclui registradores MMX específicos. Em vez disso, o processador usa uma técnica de mapeamento (Figura 12.24). Os registradores de ponto flutuante existentes são usados para armazenar valores MMX. Especificamente, 64 bits de baixa ordem (mantissa) de cada registrador de ponto flutuante são usados para formar oito registradores MMX. Assim, a antiga arquitetura x86 de 32 bits é facilmente estendida para suportar a capacidade MMX. Algumas das principais características do uso MMX desses registradores são as seguintes:

- Lembre que registradores de ponto flutuante são tratados como uma pilha para operações de ponto flutuante. Para operações MMX, esses mesmos registradores são acessados diretamente.
- Na primeira vez que uma instrução MMX é executada depois de qualquer operação de ponto flutuante, a palavra de marcação FP (do inglês *floating point*) é marcada como válida. Isso reflete na mudança de operação de pilha para endereçamento direto de registradores.
- Instrução EMMS (estado MMX vazio) define bits da marcação da palavra FP para indicar que todos os registradores estão vazios. É importante que o programador insira essa instrução no final de bloco de código MMX para que as operações de ponto flutuante subsequentes funcionem corretamente.
- Quando um valor é escrito para um registrador MMX, bits [79:64] do registrador FP correspondente (bits de expoente e de sinal) são todos definidos para o valor 1. Isso define o valor no registrador FP para NN (não numérico) ou infinito quando visto como um valor de ponto flutuante. Isso garante que um valor de dados MMX não se pareça com um valor de ponto flutuante válido.



Processamento de interrupção

O processamento de interrupção dentro de um processador é uma facilidade oferecida para suportar o sistema operacional. Isso permite que uma aplicação seja suspensa para que uma variedade de condições de interrupção possa ser atendida e depois seja reiniciada.

Figura 12.24 Mapeamento de registradores MMX para registradores de ponto flutuante

INTERRUPÇÕES E EXCEÇÕES Duas classes de eventos fazem com que o x86 suspenda a execução da corrente da instrução corrente e responda ao evento: interrupções e exceções. Em ambos os casos, o processador salva o contexto do processo atual e transfere para uma rotina predefinida atender a condição. Uma interrupção é gerada por um sinal de hardware e pode ocorrer um número aleatório de vezes durante a execução de um programa. Uma *exceção* é gerada a partir do software e é provocada pela execução de uma instrução. Existem duas origens das interrupções e duas origens das exceções:

1. Interrupções

- **Interrupções mascaráveis:** recebidas no pino INTR do processador. O processador não reconhece uma interrupção mascarável se o flag habilitar interrupção (IF) não estiver definido.
- **Interrupções não mascaráveis:** recebidas no pino NMI do processador. Reconhecimento de tais interrupções não pode ser evitado.

2. Exceções

- **Exceções detectadas pelo processador:** resultam quando processador encontra um erro enquanto tenta executar uma instrução.
- **Exceções programadas:** essas são as instruções que geram uma exceção (por exemplo, INTO, INT3, INT e BOUND).

TABELA DE VETORES DE INTERRUPÇÕES O processamento de interrupção em x86 usa tabela de vetor de interrupções. Cada tipo de interrupção possui um número vinculado e esse número é usado para indexar a tabela de vetor de interrupções. Essa tabela contém 256 vetores de interrupção de 32 bits, que representa o endereço (segmento e offset) da rotina para atender a interrupção para esse determinado número de interrupção.

A Tabela 12.3 mostra a atribuição de números na tabela de vetores de interrupções; os campos sombreados representam interrupções, enquanto os não sombreadas são exceções. A interrupção NMI de hardware é do tipo 2. Às interrupções INTR de hardware são atribuídos os números do intervalo de 32 até 255; quando uma interrupção INTR é gerada, ela precisa ser acompanhada dentro do barramento pelo número de vetor de interrupção para essa interrupção. Os números de vetores restantes são usados para exceções.

Se mais do que uma exceção ou interrupção está pendente, o processador as atende de maneira previsível. A posição de números do vetor dentro da tabela não reflete a prioridade. Em vez disso, a prioridade entre exceções e interrupções é organizada em cinco classes. Em ordem descendente de prioridade aqui estão:

Tabela 12.3 Tabela de vetores de interrupções e exceções para x86

Número do vetor	Descrição
0	Erro de divisão; estouro de divisão (<i>overflow</i>) ou divisão por zero
1	Exceção de depuração; inclui várias falhas e traps relacionadas com depuração
2	Interrupção do pino NMI; sinal no pino NMI
3	<i>Breakpoint</i> ; causado pela instrução INT 3 que é uma instrução de 1 byte útil para depuração
4	<i>Overflow</i> detectado em INTO; ocorre quando o processador executa INTO e o flag com valor 1
5	Limite excedido em BOUND excedido; instrução BOUND compara um registrador com limites armazenados na memória e gera uma interrupção se o conteúdo do registrador está fora dos limites
6	<i>Opcode</i> indefinido
7	Dispositivo indisponível; tentativa de uso da instrução ESC ou WAIT falha por causa da demora do dispositivo externo
8	Falha dupla; duas interrupções ocorrem durante a mesma instrução e não podem ser tratadas em série
9	Reservado
10	Segmento de estado de tarefa inválido; segmento que descreve a tarefa requerida não é inicializado ou não é válido
11	Segmento ausente; segmento requerido não está presente
12	Falha de pilha; limite do segmento da pilha excedido ou segmento da pilha ausente
13	Proteção geral; violação da proteção que não causa outra exceção (por exemplo, escrever no segmento que é somente para leitura)
14	Falha de página
15	Reservado
16	Erro de ponto flutuante; gerado por uma instrução aritmética de ponto flutuante
17	Verificação de alinhamento; acesso a uma palavra armazenada em um endereço de byte ímpar ou uma palavra dupla armazenada em um endereço não múltiplo de 4
18	Verificação de máquina; específico para cada modelo
19-31	Reservado
32-255	Vetores de interrupções de usuário; fornecido quando sinal INTR é ativado

Sem sombra: exceções.

Com sombra: interrupções.

- **Classe 1:** paradas (*traps*) na instrução anterior (vetor número 1).
- **Classe 2:** interrupções externas (2, 32-255).
- **Classe 3:** falhas na busca da próxima instrução (3, 14).
- **Classe 4:** falhas na decodificação da próxima instrução (6, 7).
- **Classe 5:** falhas na execução de uma instrução (0, 4, 5, 8, 10-14, 16, 17).

TRATAMENTO DE INTERRUPÇÃO Assim como acontece com a transferência da execução usando uma instrução CALL, uma transferência para uma rotina de tratamento de interrupção usa a pilha do sistema para armazenar o estado do processador. Quando ocorre uma interrupção e é reconhecida pelo processador, uma sequência de eventos acontece:

1. Se a transferência envolve uma mudança do nível de privilégio, então o registrador atual de segmento de pilha e o registrador atual ponteiro estendido de pilha (ESP) são colocados na pilha.

2. O valor atual do registrador EFLAGS é colocado na pilha.
3. Flags de interrupção (IF) e trap (TF) são definidos com valor zero. Isso desabilita interrupções INTR e a *trap* ou recurso de passo único.
4. Ponteiro de segmento de código corrente (CS) e ponteiro da instrução corrente (IP ou EIP) são colocados na pilha.
5. Se a interrupção é acompanhada por um código de erro, então o código de erro é colocado na pilha.
6. O conteúdo do vetor de interrupção é obtido e carregado nos registradores CS e IP ou EIP. A execução continua a partir da rotina que atende a interrupção.

Para retornar de uma interrupção, a rotina que atende a interrupção executa uma instrução IRET. Isso faz com que todos os valores salvos na pilha sejam recuperados; a execução continua a partir do ponto da interrupção.



12.6 Processador ARM

Nesta seção analisamos alguns elementos-chave da arquitetura e organização ARM. Deixamos a discussão de aspectos da organização e pipeline mais complexos para Capítulo 14. Para a discussão desta seção e do Capítulo 14, é útil ter em mente as principais características da arquitetura ARM. ARM é, em primeiro lugar, um sistema RISC com os seguintes características principais:

- Um conjunto moderado de registradores uniformes, mais do que são encontrados em alguns sistemas CISC, porém menos do que encontrados em muitos sistemas RISC.
- Modelo carregar/armazenar (*load/store*) de processamento de dados, no qual as operações executam apenas com os operandos nos registradores e não diretamente na memória. Todos os dados precisam ser carregados em registradores antes que uma operação possa ser efetuada; o resultado então pode ser usado para o processamento posterior ou armazenado em memória.
- Uma instrução uniforme de tamanho fixo de 32 bits para o conjunto padrão e 16 bits para o conjunto de instruções Thumb.
- Para tornar cada instrução de processamento de dados mais flexível, um deslocamento ou uma rotação pode pré-processar um dos registradores de origem. Para suportar esse recurso eficientemente, a unidade aritmética lógica (ALU) e unidades de deslocamento são separadas.
- Um número pequeno de modos de endereçamento com todos os endereços de carga/armazenamento sendo determinados a partir dos registradores e campos da instrução. Endereçamento indireto ou indexado envolvendo valores na memória não é usado.
- Modos de endereçamento com autoincremento e autodecremento são usados para melhorar a operação de laços de repetição dos programas.
- Execução condicional das instruções minimiza a necessidade das instruções de desvios condicionais, melhorando assim a eficiência do pipeline, porque a limpeza do pipeline é reduzida.

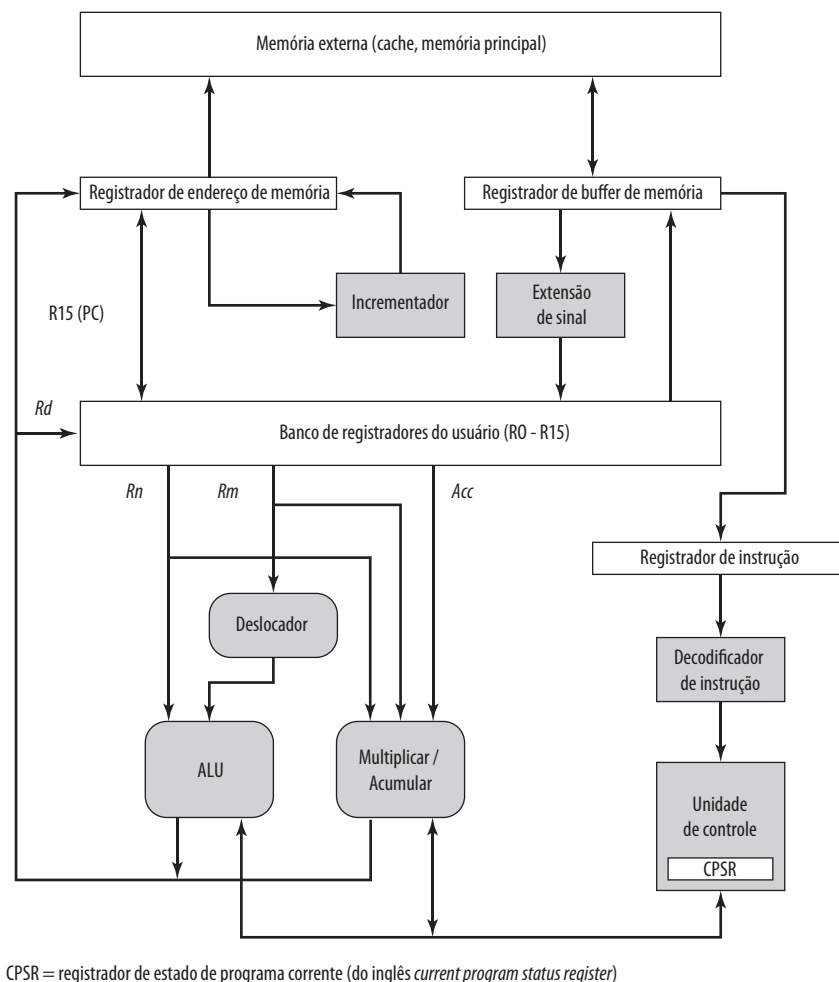


Organização do processador

A organização do processador ARM varia substancialmente de uma implementação para outra, particularmente quando são usadas diferentes versões da arquitetura ARM. No entanto, é útil para a discussão desta seção apresentar uma organização ARM simplificada e genérica, a qual é ilustrado na Figura 12.25. Nessa figura as setas indicam o fluxo de dados. Cada caixa apresenta uma unidade de hardware funcional ou uma unidade de armazenamento.

Dados são trocados com o processador a partir da memória externa por meio de um barramento de dados. O valor transferido é um item de dados, como resultado de uma instrução de carregar (*load*) ou armazenar (*store*), ou uma leitura de instrução. Instruções obtidas passam por um decodificador de instruções antes da execução, sob controle de uma unidade de controle. O último inclui lógica de pipeline e fornece sinais de controle (não mostrados) para todos os elementos de hardware do processador. Itens de dados são colocados no banco de registradores (*register file*) que consiste de um conjunto de registradores de 32 bits. Itens do tamanho de um byte ou meia palavra tratados como números de complemento de dois são estendidos com sinal até 32 bits.

As instruções de processamento de dados ARM normalmente têm dois registradores de origem, *Rn* e *Rm*, e um resultado único ou registrador de destino, *Rd*. Os valores dos registradores de origem alimentam ALU ou uma unidade de multiplicação separada que usa um registrador adicional para acumular resultados parciais. O processador ARM inclui também uma unidade de hardware que pode deslocar ou rotacionar o valor de *Rm* antes de entrar em

Figura 12.25 Organização ARM simplificada

ALU. Esse deslocamento ou rotação ocorre dentro do tempo de ciclo da instrução e aumenta o poder e a flexibilidade de muitas operações de processamento de dados.

Os resultados de uma operação são retornados para o registrador de destino. As instruções de *load/store* também podem usar a saída das unidades aritméticas para gerar o endereço de memória para carga ou armazenamento.



Modos do processador

É comum para um processador suportar apenas um número pequeno de modos do processador. Por exemplo, muitos sistemas operacionais usam apenas dois modos: modo usuário e modo kernel, este último sendo usado para executar software de sistema privilegiado. Ao contrário disso, a arquitetura ARM permite uma base flexível para que os sistemas operacionais possam impor uma variedade de políticas de proteção.

A arquitetura ARM suporta sete modos de execução. A maioria das aplicações executa em **modo usuário**. Enquanto o processador está no modo usuário, o programa sendo executado é incapaz de acessar os recursos protegidos do sistema ou de alterar o modo, causando uma exceção nesse caso.

Os seis modos de execução restantes são referidos como modos privilegiados. Esses modos são usados para executar o software de sistema. Existem duas vantagens principais para definir tantos modos privilegiados diferentes: (1) O SO pode adequar o uso do software de sistema para uma variedade de circunstâncias e (2) certos registradores são dedicados para uso para cada um dos modos privilegiados, permitindo mudanças mais rápidas no contexto.

Os modos de exceção têm acesso total aos recursos do sistema e podem mudar os modos livremente. Cinco desses modos são conhecidos como modos de exceção. Eles são ativados quando exceções específicas ocorrem.

Cada um desses modos possui alguns registradores dedicados que substituem alguns registradores do modo usuário e que são usados para evitar a corrupção das informações de estado do modo de usuário quando exceção acontece. Os modos de exceção são os seguintes:

- **Modo supervisor:** normalmente é o modo em que executa o SO. Ele é ativado quando o processador encontra uma instrução de interrupção de software. Interrupções de software são um jeito padrão para invocar os serviços do sistema operacional no ARM.
- **Modo de abortamento:** ativado como resposta a falhas de memória.
- **Modo indefinido:** ativado quando o processador tenta executar uma instrução que não é suportada nem pelo núcleo principal nem por um dos coprocessadores.
- **Modo de interrupção rápido:** ativado sempre que o processador recebe um sinal de interrupção a partir de uma fonte designada de interrupção rápida. Uma interrupção rápida não pode ser interrompida, porém uma interrupção rápida pode interromper uma interrupção normal.
- **Modo de interrupção:** ativado sempre que o processador recebe um sinal de interrupção a partir de qualquer outra origem de interrupção (diferente da interrupção rápida). Uma interrupção apenas pode ser interrompida por uma interrupção rápida.

O modo privilegiado restante é o **modo de sistema**. Este não é ativado por nenhuma exceção e usa os mesmos registradores disponíveis no modo usuário. O modo de sistema é usado para executar certas tarefas privilegiadas do sistema operacional. As tarefas do modo de sistema podem ser interrompidas por qualquer uma das cinco categorias de exceções.



Organização dos registradores

A Figura 12.26 ilustra os registradores visíveis ao usuário para ARM. O processador ARM possui um total de 37 registradores de 32 bits, classificados como segue:

- 31 registradores referenciados no manual de ARM como sendo registradores de propósito geral. Na verdade, alguns deles, como contadores de programa, possuem propósitos específicos.
- Seis registradores de estado de programa.

Registradores são arranjados em bancos parcialmente sobrepostos, sendo que o modo atual do processador define qual banco está disponível. A qualquer momento, 16 registradores numerados e um ou dois registradores de estado de programa estão visíveis, para um total de 17 ou 18 registradores visíveis ao software. A Figura 12.26 deve ser interpretada da seguinte forma:

- Registradores de R0 a R7, registrador R15 (contador de programa) e registrador de estado de programa corrente (CPSR) são visíveis e compartilhados por todos os modos.
- Registradores de R8 até R12 são compartilhados por todos os modos exceto interrupção rápida, a qual possui seus próprios registradores dedicados de R8_fiq até R12_fiq.
- Todos os modos de exceção têm suas próprias versões de registradores R13 e R14.
- Todos os modos de exceção têm um registrador próprio de estado de programa dedicado salvo (SPSR, do inglês *saved program status register*)

REGISTRADORES DE PROPÓSITO GERAL O registrador R13 é normalmente usado como ponteiro de pilha e é conhecido também como SP (*Stack points*). Como cada modo de exceção possui um R13 separado, cada modo de exceção pode ter a sua própria dedicada pilha de programa. R14 é conhecido como registrador de ligação (LR — *link register*) e é usado para guardar endereço de retorno da sub-rotina e retornos do modo de exceção. O registrador R15 é o contador de programas (PC — *Program Counter*).

REGISTRADORES DE ESTADOS DE PROGRAMA O CPSR é acessível em todos os modos do processador. Cada modo de exceção também possui um SPSR dedicado que é usado para preservar o valor de CPSR quando a exceção associada acontece.

Os 16 bits mais significativos do CPSR contêm flags de usuário visíveis no modo usuário e que podem ser usadas para afetar a operação de um programa (Figura 12.27). Aqui estão elas:

- **Flags de código condicional:** flags N, Z, C e V, discutidas no Capítulo 10.
- **Flag Q:** usado para indicar se um *overflow* e/ou saturação ocorreu em alguma instrução orientada a SIMD.
- **Bit J:** indica o uso de instruções especiais de 8 bits, conhecidas como instruções Jazelle e que estão fora do escopo do nosso estudo.

Figura 12.26 Organização dos registradores do ARM

Modos						
Modos privilegiados						
Modos de exceção						
Usuário	Sistema	Supervisor	Abortamento	Indefinido	Interrupção	Interrupção rápida
R0	R0	R0	R0	R0	R0	R0
R1	R1	R1	R1	R1	R1	R1
R2	R2	R2	R2	R2	R2	R2
R3	R3	R3	R3	R3	R3	R3
R4	R4	R4	R4	R4	R4	R4
R5	R5	R5	R5	R5	R5	R5
R6	R6	R6	R6	R6	R6	R6
R7	R7	R7	R7	R7	R7	R7
R8	R8	R8	R8	R8	R8	R8_fiq
R9	R9	R9	R9	R9	R9	R9_fiq
R10	R10	R10	R10	R10	R10	R10_fiq
R11	R11	R11	R11	R11	R11	R11_fiq
R12	R12	R12	R12	R12	R12	R12_fiq
R13(SP)	R13(SP)	R13_svc	R13_abtR	13_und	R13_irq	R13_fiq
R14(LR)	R14(LR)	R14_svc	R14_abt	R14_und	R14_irq	R14_fiq
R15(PC)	R15(PC)	R15(PC)	R15(PC)	R15(PC)	R15(PC)	R15(PC)

CPSR	CPSR	CPSR	CPSR	CPSR	CPSR	CPSR
		SPSR_svc	SPSR_abt	SPSR_und	SPSR_irq	SPSR_fiq

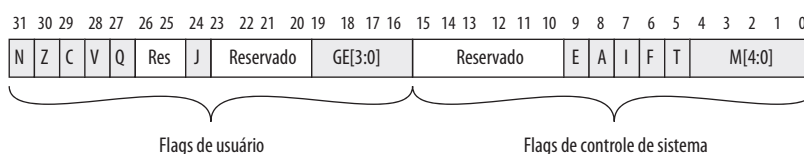
Sombreado indica que o registrador normal usado pelo modo usuário ou de sistema foi substituído por um registrador específico para modo de exceção.

SP = ponteiro de pilha CPSR = registrador de estado de programa corrente
LR = registrador de ligação SPSR = registrador de estado de programa salvo
PC = contador de programa

- **Bits GE[3:0]:** instruções SIMD usam bits[19:16] como flags maior que ou igual (GE — *greater than or equal*) para bytes individuais ou meias palavras do resultado.

Os 16 bits menos significantes de CPSR contêm flags de controle de sistema que podem ser alterados apenas quando o processador está num modo privilegiado. Os campos estão descritos a seguir:

Figura 12.27 Formato de CPSR e SPSR de ARM



- **Bit E:** controla carga e armazenamento *endianness* de dados; ignorado para leitura de instruções.
- **Bits para desabilitar interrupção:** o bit A, quando definido com valor 1, desabilita o abortamento impreciso de dados; bit I, quando definido com valor 1, desabilita interrupções IRQ; e bit F, quando definido com valor 1, desabilita interrupções FIQ.
- **Bit T:** indica se a instrução deve ser interpretada como uma instrução ARM normal ou uma instrução Thumb.
- **Bits de modo:** indica o modo do processador.



Processamento de interrupção

Como em qualquer outro processador, o ARM inclui um dispositivo que permite ao processador interromper o programa corrente para lidar com condições de exceção. Exceções são geradas por fontes internas e externas para fazer com que o processador trate um evento. O estado do processador logo antes de tratar uma exceção é normalmente preservado para que o programa original possa ser reiniciado quando a rotina de exceção esteja completa. Mais de uma exceção pode ocorrer ao mesmo tempo. A arquitetura ARM suporta sete tipos de exceções. A Tabela 12.4 lista os tipos de exceções e o modo do processador que é usado para processar cada tipo. Quando uma exceção ocorre, a execução é forçada de um endereço fixo de memória correspondente ao tipo de exceção. Esses endereços fixos são chamados de vetores de exceção.

Se mais do que uma interrupção está aguardando, elas são tratadas em ordem de prioridade. A Tabela 12.4 mostra as exceções em ordem de prioridade, das mais altas até as mais baixas.

Quando ocorre uma exceção, o processador para a execução depois da instrução corrente. O estado do processador é preservado em SPSR que corresponde ao tipo de exceção, para que o programa original possa ser reiniciado quando a rotina de exceção estiver completa. O endereço da instrução que o processador estava para executar é colocado no registrador de ligação do modo do processador apropriado. Para retornar depois do tratamento da exceção, SPSR é movido para dentro de CPSR e R14 é movido para dentro de PC.

Tabela 12.4 Vetor de interrupção ARM

Tipo de exceção	Modo	Endereço normal de entrada	Descrição
Reset	Supervisor	0x00000000	Ocorre quando o sistema é inicializado
Abortar dados	Abortamento	0x00000010	Ocorre quando um endereço de memória inválido foi acessado, como se não houvesse memória física para o endereço ou falta permissão correta de acesso.
FIQ (interrupção rápida)	FIQ	0x0000001C	Ocorre quando um dispositivo externo ativa o pino FIQ no processador. Uma interrupção não pode ser interrompida exceto por uma FIQ. A FIQ é projetada para suportar uma transferência de dados ou processo de canal e tem registradores privados suficientes para tirar a necessidade de salvar os registradores em tais aplicações, economizando a sobrecarga da troca de contexto. Uma interrupção rápida não pode ser interrompida.
IRQ (interrupção)	IRQ	0x00000018	Ocorre quando um dispositivo externo ativa o pino IRQ do processador. Uma interrupção não pode ser interrompida, exceto por uma FIQ.
Abortamento de busca antecipada	Abortamento	0x0000000C	Ocorre quando uma tentativa de obter uma instrução resulta em uma falha de memória. A exceção é levantada quando a instrução entra no estágio de execução do pipeline.
Instrução indefinida	Indefinido	0x00000004	Ocorre quando uma instrução que não esteja no conjunto de instruções atinge o estágio de execução do pipeline
Interrupção de software	Supervisor	0x00000008	Geralmente usado para permitir que os programas do modo usuário chamem o SO. O programa de usuário executa uma instrução SWI com um argumento que identifica a função que usuário quer executar.



12.7 Leitura recomendada

Patt (2001^o) e Moshovos e Sohi (2001^p) fornecem uma cobertura excelente de questões de pipeline discutidas neste capítulo. Hennessy e Joupi (1991^a) contém uma discussão detalhada sobre pipeline. Sohi (1990^o) fornece uma discussão excelente e detalhada sobre questões de projeto de hardware envolvidas em um pipeline de instruções. Ramamoorthy (1977^s) é um artigo clássico sobre o assunto e ainda vale muito a leitura.

Evers e Yeh (2001^t) examina a evolução de estratégias de previsão de desvios. Cragon (1992^u) é um estudo detalhado sobre previsão de desvios e pipelines de instruções. Dubey e Flynn (1991^v) e Lilja (1988^k) examinam várias estratégias de previsão de desvios que podem ser usadas para melhorar o desempenho de pipeline de instruções. Kaeli e Emma (1991^w) analisa a dificuldade introduzida na previsão de desvios pelas instruções cujo endereço do alvo é variável.

Brey (2009^x) fornece uma boa cobertura de processamento de interrupções em x86. Fog (2008^y) fornece uma discussão detalhada sobre arquitetura de pipeline para família x86.

Principais termos, perguntas de revisão e problemas

Principais termos

Previsão de desvios	Flag	Busca antecipada de instrução
Código condicional	Ciclo da instrução	Palavra de estado de programa (PSW)
Desvio atrasado	Pipeline de instrução	

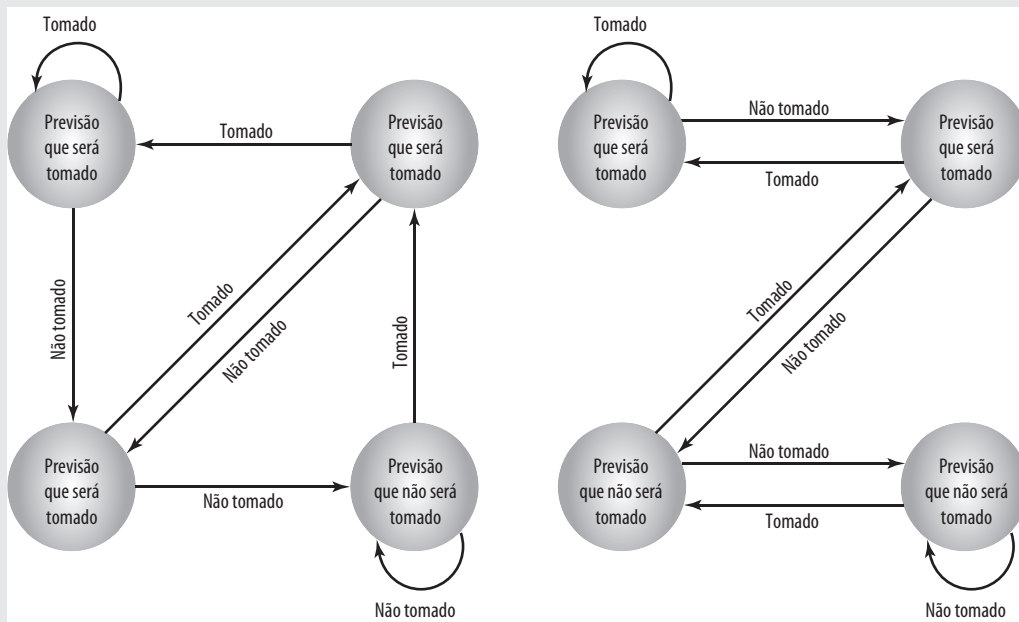
Perguntas de revisão

- 12.1 Quais papéis gerais são desempenhados pelos registradores do processador?
- 12.2 Quais categorias de dados são normalmente suportadas pelos registradores visíveis ao usuário?
- 12.3 Qual é a função de códigos condicionais?
- 12.4 O que é uma palavra de estado do programa?
- 12.5 Por que um pipeline de instrução de dois estágios dificilmente diminuirá o tempo do ciclo da instrução pela metade, quando comparado a um sistema sem pipeline?
- 12.6 Liste e explique resumidamente várias formas em que um pipeline de instruções pode lidar com instruções de desvio condicional.
- 12.7 Como são usados os bits de histórico para previsão de desvios?

Problemas

- 12.1 a. Se a última operação executada em um computador com uma palavra de 8 bits foi uma adição em que dois operandos eram 00000010 e 00000011, qual seria o valor dos seguintes flags?
 - Carry.
 - Zero.
 - Overflow.
 - Sinal.
 - Paridade igual.
 - Half-Carry (carry auxiliar).
- b. Repita para adição de -1 (complemento a dois) e $+1$
- 12.2 Repita o Problema 12.1 para operação $A - B$, onde A contém 11110000 e B contém 0010100.
- 12.3 Um microprocessador tem um clock de 5 Ghz.
 - a. Quanto tempo leva um ciclo de clock?
 - b. Qual é a duração de um tipo particular de instrução de máquina que consiste de três ciclos de clock?

- 12.4** Um microprocessador fornece uma instrução capaz de mover uma cadeia de bytes de uma área de memória para outra. A leitura e decodificação inicial da instrução levam 10 ciclos de clock. Depois demora 15 ciclos de clock para transferir cada byte. O microprocessador possui um clock de 10 GHz.
- Determine o tamanho do ciclo da instrução para o caso de uma cadeia de 64 bytes.
 - Qual é o pior atraso para aceitar uma interrupção se a instrução não puder ser interrompida?
 - Repita a parte (b) assumindo que a instrução possa ser interrompida no começo da transferência de cada byte.
- 12.5** O Intel 8088 consiste de uma unidade de interface de barramento (UIB) e uma unidade de execução (UE), o que forma um pipeline de dois estágios. A UIB obtém as instruções para uma fila de instruções de 4 bytes. Ela também participa dos cálculos de endereço, obtém operandos e escreve os resultados na memória conforme requisitado por UE. Se nenhuma dessas requisições estiver aguardando e o barramento estiver livre, UIB preenche quaisquer folgas na fila de instruções. Quando UE completa a execução de uma instrução, ela passa quaisquer resultados para UIB (destinados para memória ou E/S) e procede com a próxima instrução.
- Suponha que tarefas executadas por UIB e UE levem o mesmo tempo. Com que fator pipeline melhora o desempenho de 8088? Ignore os efeitos de instruções de desvio.
 - Repita os cálculos assumindo que UE demore duas vezes mais que UIB.
- 12.6** Suponha que um 8088 esteja executando um programa no qual a probabilidade de um salto de programa é 0.1. Para simplificar, assuma que todas as instruções sejam do tamanho de 2 bytes.
- Qual fração do ciclo de leitura do barramento de instrução é desperdiçada?
 - Repita para fila de instrução de tamanho de 8 bytes.
- 12.7** Considere o diagrama de tempo da Figura 12.10. Suponha que exista apenas um pipeline de dois estágios (busca e execução). Redesenhe o diagrama para mostrar quantas unidades de tempo são necessárias agora para quatro instruções.
- 12.8** Suponha um pipeline com quatro estágios: busca da instrução (FI), decodificação de instrução e cálculo dos endereços (DE), busca dos operandos (FO) e executar (EX). Desenhe um diagrama semelhante à Figura 12.10 para uma sequência de 7 instruções onde a terceira instrução é um desvio que é tomado e onde não haja dependências de dados.
- 12.9** Um processador de pipeline tem um taxa de clock de 2,5 GHz e executa um programa com 1,5 milhões de instruções. O pipeline possui cinco estágios e as instruções são emitidas numa taxa de uma por ciclo de clock. Ignore penalidades por causa das instruções de desvio e execuções fora de ordem.
- Qual a diferença de velocidade deste processador para este programa comparado a um processador sem pipeline, fazendo a mesma suposição usada na Seção 12.4?
 - Qual o rendimento (em MIPS) do processador com pipeline?
- 12.10** Um processador sem pipeline tem uma taxa de clock de 2.5 GHz e um CPI (ciclos por instrução) médio de 4. Uma atualização no processador introduz um pipeline de cinco estágios. No entanto, por causa dos atrasos internos do pipeline, a taxa de clock do novo processador deve ser reduzida para 2 GHz.
- Qual o aumento de velocidade obtido para um programa típico?
 - Qual a taxa em MIPS para cada processador?
- 12.11** Considere uma sequência de instruções de tamanho n que está sendo executada pelo pipeline de instruções. Seja p a probabilidade de encontrar um desvio condicional ou incondicional e seja q a probabilidade de a execução da instrução de desvio i causar um salto para um endereço não consecutivo. Assuma que cada salto desses requer que o pipeline seja esvaziado, destruindo todo o processamento das instruções, quando i emerge do último estágio. Revise as equações 12.1 e 12.2 para levar essas probabilidades em conta.
- 12.12** Uma limitação da abordagem de múltiplos fluxos para lidar com desvios em um pipeline é que desvios adicionais serão encontrados antes que o primeiro desvio seja resolvido. Sugira mais duas limitações ou desvantagens.
- 12.13** Considere os diagramas de estado da Figura 12.28.
- Descreva o comportamento de cada um deles.
 - Compare-os com o diagrama de estado de previsão de desvios da Seção 12.4. Discuta os méritos relativos de cada uma das três abordagens de previsão de desvios.

Figura 12.28 Dois diagramas de estado de previsão de desvios

12.14 Máquinas Motorola 680x0 incluem a instrução Decrementar e Desvio de Acordo com Condição, a qual tem a seguinte forma: DBcc Dn, displacement

onde cc é uma das condições testáveis, Dn é um registrador de uso geral e *displacement* (deslocamento) especifica o endereço alvo relativo ao endereço atual. A instrução pode ser definida da seguinte forma:

```
if (cc = False)
then begin
    Dn := (Dn) - 1;
    if Dn ≠ -1 then PC := (PC) + displacement end
else PC := (PC) + 2;
```

Quando a instrução é executada, a condição é primeiramente testada para determinar se a condição de término para o laço é satisfeita. Se for, nenhuma operação é executada e a execução continua na próxima instrução da sequência. Se a condição for falsa, o registrador de dados específico é decrementado e verificado para ver se é menor que zero. Se for menor que zero, o laço é terminado e a execução continua na próxima instrução da sequência. Caso contrário, o programa desvia para a posição especificada. Considere agora o seguinte fragmento de um programa na linguagem de montagem:

```
AGAIN    CMPM.L    (A0)+,(A1) +
         DBNE      D1, AGAIN
         NOP
```

Duas *strings* endereçadas por A0 e A1 são comparadas pela igualdade; os ponteiros de *string* são incrementados em cada referência. D1 inicialmente contém o número de palavras grandes (4 bytes) para serem comparadas.

- O conteúdo inicial dos registradores é A0 = \$00004000, A1 = \$00005000 e D1 = \$000000FF (\$ indica notação hexadecimal). A memória entre \$4000 e \$6000 é preenchida com palavras \$AAAA. Se o programa acima mencionado for executado, especifique o número de vezes que o laço DBNE é executado e o conteúdo dos três registradores quando a instrução NOP é alcançada.
- Repita (a), mas suponha agora que memória entre \$4000 e \$4FEE é preenchida com \$0000 e entre \$5000 e \$6000 é preenchida com \$AAA.

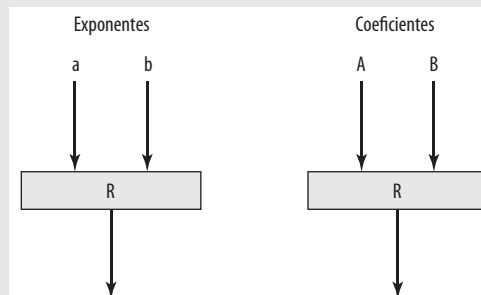
12.15 Redesenhe a Figura 12.19c, supondo que o desvio condicional não seja tomado.

12.16 A Tabela 12.5 sumariza estatísticas de MacDougall (1984^h) com respeito ao comportamento de desvios para várias classes de aplicações. Com exceção do tipo 1 para comportamento de desvios não há diferença notável entre os tipos de aplicações. Determine a fração de todos os desvios que vão para o endereço alvo do desvio para o ambiente científico. Repita para ambientes comerciais e de sistemas.

Tabela 12.5 Comportamento de desvios em exemplos de aplicações

Ocorrências de classes de desvios			
Tipo 1: Desvio 72,5%			
Tipo 2: Controle de laço 9,8%			
Tipo 3: Chamada de procedimento, retorno 17,7%			
Desvio tipo 1: para onde vai	Científico	Comercial	Sistemas
Incondicional – 100% vai para alvo	20%	40%	35%
Condicional – foi para alvo	43,2%	24,3%	32,5%
Condicional – não foi para alvo	36,8%	35,7%	32,5%
Desvio tipo 2 (todos os ambientes)			
Vai para alvo		91%	
Não vai para alvo		9%	
Desvio tipo 3			
100% vai para alvo			

- 12.17** O pipelining pode ser aplicado dentro de ALU para acelerar operações de ponto flutuante. Considere o caso de adição e subtração de ponto flutuante. Em termos simplificados, pipeline poderia ter quatro estágios: (1) Comparar expoentes; (2) Escolher o expoente e alinhar os coeficientes; (3) Adicionar ou subtrair coeficientes; (4) Normalizar os resultados. O pipeline pode ser considerado como tendo duas *threads* paralelas, uma tratando dos expoentes e outra tratando dos coeficientes, e poderia começar desta forma:



Nesta figura, as caixas nomeadas R se referem a um conjunto de registradores usados para guardar resultados temporários. Complete o diagrama de blocos que mostra a estrutura do pipeline em nível mais alto.

Referências

- a LUNDE, A. "Empirical evaluation of some features of instruction set processor architectures". *Communications of the ACM*, mar. 1977.
- b WILLIAMS, F. e STEVEN, G. "Address and data register separation on the M68000 family". *Computer Architecture News*, jun. 1990.
- c DeROSA, J. e LEVY, H. "An evaluation of branch architectures". *Proceedings, Fourteenth Annual International Symposium on Computer Architecture*, 1987.
- d STRITTER, E. e GUNTER, T. "A microprocessor architecture for a changing world: the Motorola 68000". *Computer*, fev. 1979.
- e MORSE, S.; POHLMAN, W. e RAVENEL, B. "The Intel 8086 microprocessor: a 16-bit evolution of the 8080". *Computer*, jun. 1978.
- f TOONG, H. e GUPTA, A. "An architectural comparison of contemporary 16-bit microprocessors". *IEEE Micro*, mai. 1981.
- g El-Ayat, K. e AGARWAL, R. "The Intel 80386—Architecture and implementation". *IEEE Micro*, dez. 1985.
- h MACDOUGALL, M. "Instruction-level program and process modeling". *IEEE Computer*, jul. 1984.
- i ANDERSON, D.; SPARACIO, F. e TOMASULO, F. "The IBM system/360 model 91: machine philosophy and instruction handling". *IBM Journal of Research and Development*, jan. 1967.
- j HWANG, K. *Advanced computer architecture*. Nova York: McGraw-Hill, 1993.

- k LILJA, D. "Reducing the branch penalty in pipelined processors". *Computer*, jul. 1988.
- l RECHES, S. e WEISS, S. "Implementation and analysis of path history in dynamic branch prediction schemes". *IEEE Transactions on Computers*, ago. 1998.
- m YEH, T. e PATT, N. "Two-level adapting training branch prediction". *Proceedings, 24th Annual International Symposium on Microarchitecture*, 1991.
- n CRAWFORD, J. "The i486 CPU: executing instructions in one clock cycle". *IEEE Micro*, fev. 1990.
- o PATT, Y. "Requirements, bottlenecks, and good fortune: agents for microprocessor evolution". *Proceedings of the IEEE*, nov. 2001.
- p MOSHOVOS, A. e SOHI, G. "Microarchitectural innovations: boosting microprocessor performance beyond semiconductor technology scaling". *Proceedings of the IEEE*, nov. 2001.
- q HENNESSY, J. e JOUPPI, N. "Computer technology and architecture: an evolving interaction". *Computer*, set. 1991.
- r SOHI, G. "Instruction issue logic for high-performance interruptable, multiple functional unit, pipelined computers". *IEEE Transactions on Computers*, mar. 1990.
- s RAMAMOORTHY, C. "Pipeline architecture". *Computing Surveys*, mar. 1977.
- t EVERS, M. e YEH, T. "Understanding branches and designing branch predictors for high-performance microprocessors". *Proceedings of the IEEE*, nov. 2001.
- u CRAGON, H. *Branch strategy taxonomy and performance models*. Los Alamitos, CA: IEEE Computer Society Press, 1992.
- v DUBEY, P. e FLYNN, M. "Branch strategies: modeling and optimization". *IEEE Transactions on Computers*, out. 1991.
- w KAEI, D. e EMMA, P. "Branch history table prediction of moving target branches due to subroutine returns". *Proceedings, 18th Annual International Symposium on Computer Architecture*, maio 1991.
- x BREY, B. *The Intel microprocessors: 8086/8066, 80186/80188, 80286, 80386, 80486, Pentium, Pentium Pro Processor, Pentium II, Pentium III, Pentium 4 and Core2 with 64-bit extensions*. Upper Saddle River, NJ: Prentice Hall, 2009.
- y FOG, A. *The microarchitecture of Intel and AMD CPUs*. Copenhagen University College of Engineering, 2008. Disponível em: <<http://www.agner.org/optimize/>>.